

# Towards Fast and Safety-Guaranteed Trajectory Planning and Tracking for Time-Varying Systems

Seth Siriya, Mo Chen, and Ye Pu

**Abstract**—When deploying autonomous systems in unknown and changing environments, it is critical that their motion planning and control algorithms are computationally efficient and can be reapplied online in real time, whilst providing theoretical safety guarantees in the presence of disturbances. The satisfaction of these objectives becomes more challenging when considering time-varying dynamics and disturbances, which arise in real-world contexts. We develop methods with the potential to address these issues by applying an offline-computed safety guaranteeing controller on a physical system, to track a virtual system that evolves through a trajectory that is replanned online, accounting for constraints updated online. The first method we propose is designed for general time-varying systems over a finite horizon. Our second method overcomes the finite horizon restriction for periodic systems. We simulate our algorithms on a case study of an autonomous underwater vehicle subject to wave disturbances.

**Index Terms**—Nonlinear control systems, real-time systems, safety, time-varying systems.

## I. INTRODUCTION

Robotic systems have demonstrated great success in a wide range of applications, including self-driving cars in urban environments [1], unmanned autonomous vehicles (UAVs) for disaster response [2], and autonomous underwater vehicles (AUVs) [3] for seafloor mapping. As they become more commonplace and integrated into society, it is pertinent that they can efficiently plan trajectories to safely maneuver through a priori unknown and changing environments in real time. One of the major challenges to further widespread adoption of these technologies are complex, time-varying, system dynamics and disturbances, which hinder the safety of planned trajectories. This motivates the need to develop strategies for safe navigation of nonlinear systems with time-varying dynamics and bounded disturbances capable of handling constraints updated online in real time.

Various approaches to safe planning and control have been considered in recent years. In [4], a finite library of open-loop trajectories and feedback controllers are precomputed offline, and then sum-of-squares (SOS) optimization is used to compute a safe *funnel*. Subsequent SOS-based methods remove the finite library requirement to consider either a parameterized class of, or arbitrary, planned trajectories (e.g. [5]–[9]). Another method is control barrier

functions (CBFs) [10], which provide inequality constraints on control inputs that guarantee the system remains in a safe set. CBFs for low-level control have been combined with model predictive control (MPC) for high-level planning in [11]. Control contraction metrics for low-level control have been applied to track a nominal reference trajectory computed using MPC, with guarantees that the system remains in a tube around the nominal reference [12], bearing similarities to tube-based MPC [13]. These results often conservatively consider worst-case additive disturbances over the state space, motivating [14] which proposes a planning strategy that accounts for disturbances parameterized over the state space. However, all of these methods have their own respective challenges, such as the need for polynomial dynamics in SOS-based methods, the difficulty of searching for CBFs for nonlinear systems, and the online computation of nominal reference trajectories in nonlinear MPC. Moreover, they do not consider time-varying dynamics.

FaSTrack [15], [16] is another method that follows a similar strategy to address the challenge of safe navigation of nonlinear systems with online constraint satisfaction and bounded disturbances. It achieves this through the offline precomputation of a safe, optimal tracking controller and associated tracking error bound (TEB) using Hamilton Jacobi (HJ) reachability. This controller is applied online to track a simple *planning system* evolving through a trajectory computed via an arbitrary computationally efficient planning method, enabling real-time performance. However, time-varying dynamics are not considered. A naive extension to safely handle time-varying dynamics is to uniformly bound the time-varying components, but this approach may be too conservative. Moreover, the issue of when or how to replan to effectively navigate through diverse environments is not thoroughly explored. Different speeds for the planning system are considered in [17] to address this, but more expensive offline computation is required.

Motivated by these challenges, we aim to develop a method for the safe navigation of *time-varying* nonlinear systems that satisfy constraints updated online in real time, despite the presence of bounded disturbances. Our approach is inspired by FaSTrack and similarly consists of an offline computation stage and an online planning and tracking stage. We started this line of work in [18], which proposed a method for real-time safe trajectory planning of AUVs in plane progressive waves — a time-varying system. Although we will use AUVs as a motivating example, unlike [18], in this paper we address the combined replanning and tracking problem for general time-varying systems. In particular, our contributions are as follows:

- 1) We propose a trajectory replanning and tracking method for the safe navigation of *general time-varying* dynamical systems to a goal while satisfying constraints that are updated online,

The work of Seth Siriya was supported by the Australian Government Research Training Program (RTP) Scholarship. The work of Ye Pu was supported by the Australian Research Council under Project DE220101527.

S. Siriya and Y. Pu are with the Department of Electrical and Electronic Engineering, University of Melbourne, Parkville, VIC 3010, Australia ({siriya@student, ye.pu}@unimelb.edu.au; +61 390355511). M. Chen is with the School of Computing Science, Simon Fraser University, Burnaby, BC V5A 1S6, Canada (mochen@cs.sfu.ca; +1 7787827198).

over a *restricted* time horizon. Our strategy shows promise for real-time performance, which is enabled via the precomputation of a value function and safe, optimal tracking controller offline based on a time-varying model of the dynamics, over this horizon. We derive theoretical guarantees for online constraint satisfaction and goal-reaching.

- 2) Our procedure in 1) allows the planning system to be freely initialized in a variably sized set based on the value function during replanning. This allows the *planning system* trajectory to exhibit discontinuous jumps during replanning, or ‘teleport’, enabling greater flexibility in performance across diverse environments.
- 3) We propose a trajectory replanning and tracking method for *periodic* systems that overcomes the time horizon restriction in 1) to achieve constraint satisfaction and goal reaching for *arbitrarily* long horizons despite the HJ precomputation being performed over a short time interval. We theoretically analyze the method.
- 4) We demonstrate our methods using a numerical example of an AUV in plane progressive waves. Firstly, we show that our time-varying procedure in 1) successfully achieves obstacle avoidance and goal reaching in a tight, *a priori* unknown environment by updating known obstacle information in real time using sensor data. This is contrasted against an alternate time-invariant procedure based on FaSTrack [16]. Secondly, we demonstrate that flexible reinitialization of the planning system can be exploited for adaptive behavior. Lastly, we demonstrate the success of the procedure from 3) in a lengthy simulation.

The remainder of the paper is organized as follows: Sec. II contains our problem formulation. In Sec. III, the offline stage of our framework is provided. In Sec. IV, the planning procedure for the online stage is detailed. The replanning and tracking framework for time-varying systems is provided in Sec. V, alongside a theoretical analysis of its safety guarantees. This is repeated in Sec. VI for the case of periodic systems. In Sec. VII, we provide an example AUV system model subject to wave disturbances and show how our framework can be implemented for it. Conclusions are provided in Sec. VIII. Proofs of all theoretical results are deferred to Appendix I.

## II. PROBLEM FORMULATION

In this section, we describe the problem of navigating a time-varying system to a goal while satisfying constraints updated online.

Consider a dynamical system — which we henceforth refer to as the *tracking system* — governed by the dynamics:

$$\dot{s} = f(t, s, u_s, d) \quad (1)$$

where  $f: [0, T] \times \mathcal{S} \times \mathcal{U}_s \times \mathcal{D} \rightarrow \mathbb{R}^{n_s}$  is the *tracking system model*,  $t$  is time,  $[0, T]$  is the time interval where  $f$  is defined,  $s \in \mathcal{S} \subseteq \mathbb{R}^{n_s}$  is the *tracking system state*,  $u_s$  is the *tracking system control* taking values in a compact set  $\mathcal{U}_s \subseteq \mathbb{R}^{n_u}$ , and  $d$  is the *disturbance* taking values in a compact set  $\mathcal{D} \subseteq \mathbb{R}^{n_d}$ .

Our objective is to control the tracking system starting from some initial state  $s_0$  at time  $t=0$ , so it satisfies control constraint  $\mathcal{U}_s$ , remains inside a state constraint set  $\mathcal{C} \subseteq \mathcal{S}$  which is updated online in real time and reaches a desired goal region  $\mathcal{G} \subseteq \mathbb{R}^{n_s}$ , over the interval  $t \in [0, T_{\text{run}}]$ , where  $T_{\text{run}} \leq T$  is the *task horizon* over which the system is controlled. For example,  $\mathcal{U}_s$  for an AUV could be its actuator limits,  $\mathcal{C}$  may contain velocity constraints and the complement of the obstacles to be avoided in an environment, and  $\mathcal{G}$  could be its desired

position and velocity (see Sec. VII for a specific example). This is to be achieved regardless of the disturbance  $d \in \mathcal{D}$  affecting the system.

Throughout this paper, we assume  $f$  is uniformly continuous, bounded, and Lipschitz continuous in  $s$  uniformly in  $t$ ,  $u_s$ , and  $d$ . Therefore, given control inputs  $u_s(\cdot) \in \{\phi: [t, T] \rightarrow \mathcal{U}_s: \phi(\cdot) \text{ is measurable}\}$ , disturbances  $d(\cdot) \in \{\phi: [t, T] \rightarrow \mathcal{D}: \phi(\cdot) \text{ is measurable}\}$ , and initial state  $s \in \mathcal{S}$  at time  $t \in [0, T]$ , there exists a unique continuous trajectory that solves (1) [19]. We denote this trajectory as  $\xi_f(\cdot; s, t, u_s(\cdot), d(\cdot)): [t, T] \rightarrow \mathcal{S}$ . It satisfies (1) with an initial condition almost everywhere, such that  $\frac{d}{dt}\xi_f(t'; s, t, u_s(\cdot), d(\cdot)) = f(t', \xi_f(t'; s, t, u_s(\cdot), d(\cdot)), u_s(t'), d(t'))$  for  $t' \in [t, T]$ , with  $\xi_f(t; s, t, u_s(\cdot), d(\cdot)) = s$ .

## III. OFFLINE COMPUTATION

In this section, we describe the offline stage of our method for trajectory replanning and tracking for time-varying systems. We start by describing the planning and relative systems in Sec. III-A. Following this, in Sec. III-B we formally introduce the value function and optimal tracking controller, and their computation. We also provide the sublevel set invariance guarantee associated with the controller and value function, then summarize the offline stage in Alg. 1.

### A. The Planning and Relative System

1) *Planning system*: The *planning system* is governed by the dynamics:

$$\dot{p} = h(p, u_p), \quad p \in \mathcal{P}, \quad u_p \in \mathcal{U}_p, \quad (2)$$

where  $h: \mathcal{P} \times \mathcal{U}_p \rightarrow \mathbb{R}^{n_p}$  is the *planning system model*,  $p \in \mathcal{P} \subseteq \mathbb{R}^{n_p}$  is the *planning system state*,  $u_p \in \mathcal{U}_p \subseteq \mathbb{R}^{n_{u_p}}$  is the *planning system control*, and  $\mathcal{U}_p$  is a compact set representing the planning system control constraint. We assume  $h$  is uniformly continuous, bounded, and Lipschitz continuous in  $p$  uniformly in  $u_p$ . Next, define the set of measurable functions  $\mathcal{U}_p(t) := \{\phi: [t, T_{\text{off}}] \rightarrow \mathcal{U}_p: \phi(\cdot) \text{ is measurable}\}$ . Here,  $T_{\text{off}} \in [0, T]$  denotes the *offline computation horizon*, which is the finite horizon over which the offline computation is to be performed. Then, given planning system controls  $u_p(\cdot) \in \mathcal{U}_p(t)$  and initial state  $p \in \mathcal{P}$  at time  $t \in [0, T_{\text{off}}]$ , there exists a unique continuous trajectory that solves (2) [19]. We denote this trajectory as  $\xi_h(\cdot; p, t, u_p(\cdot)): [t, T_{\text{off}}] \rightarrow \mathcal{P}$ . It satisfies (2) with an initial condition almost everywhere, such that  $\frac{d}{dt}\xi_h(t'; p, t, u_p(\cdot)) = h(\xi_h(t'; p, t, u_p(\cdot)), u_p(t'))$  for  $t' \in [t, T_{\text{off}}]$ , with  $\xi_h(t; p, t, u_p(\cdot)) = p$ .

2) *Relative system*: The *relative system model* is defined as

$$g(t, r, u_s, u_p, d) := Lf(t, s, u_s, d) - Mh(p, u_p) \quad (3)$$

for  $t \in [0, T_{\text{off}}]$ ,  $u_s \in \mathcal{U}_s$ ,  $u_p \in \mathcal{U}_p$ ,  $d \in \mathcal{D}$  and relative state  $r = Ls - Mp$ . Here,  $L$  and  $M$  are specially chosen matrices that respectively embed the tracking system and planning system states into the relative system state space  $\mathcal{R} \subseteq \mathbb{R}^{n_r}$ , where  $n_r$  is the dimensionality of the space. Note that these systems do not need to evolve in the same space, allowing for the choice of simple planning models. However, care needs to be taken when selecting  $M \in \mathbb{R}^{n_r \times n_p}$  and  $L \in \mathbb{R}^{n_r \times n_s}$  so that 1) common coordinates for the planning and tracking system are matched, forming the *tracking error*, and 2) important information in the system dynamics is not lost. The latter is not considered in FaSTrack [16], but its importance will become apparent in our example in VII-B. Now, consider the ODE

$$\dot{r} = g(t, r, u_s, u_p, d). \quad (4)$$

Define the sets of measurable functions  $\mathbb{U}_s(t) := \{\phi : [t, T_{\text{off}}] \rightarrow \mathcal{U}_s : \phi(\cdot) \text{ is measurable}\}$ , and  $\mathbb{D}(t) := \{\phi : [t, T_{\text{off}}] \rightarrow \mathcal{D} : \phi(\cdot) \text{ is measurable}\}$ . Under the assumptions imposed on (1) and (2), given  $u_s(\cdot) \in \mathbb{U}_s(t)$ ,  $u_p(\cdot) \in \mathbb{U}_p(t)$ ,  $d(\cdot) \in \mathbb{D}(t)$ , and initial relative state  $r \in \mathcal{R}$  at time  $t \in [0, T_{\text{off}}]$ , there exists a unique continuous trajectory that solves (4) [19]. We denote this trajectory as  $\xi_g(\cdot; r, t, u_s(\cdot), u_p(\cdot), d(\cdot)) : [t, T_{\text{off}}] \rightarrow \mathcal{R}$ . It satisfies (4) with an initial condition almost everywhere:

$$\begin{aligned} \frac{d}{dt'} \xi_g(t'; r, t, u_s(\cdot), u_p(\cdot), d(\cdot)) = \\ g(t', \xi_g(t'; r, t, u_s(\cdot), u_p(\cdot), d(\cdot)), u_s(t'), u_p(t'), d(t')), \quad t' \in [t, T_{\text{off}}], \\ \xi_g(t; r, t, u_s(\cdot), u_p(\cdot), d(\cdot)) = r. \end{aligned}$$

## B. The Value Function and Optimal Tracking Controller

We begin by describing the differential game, which is used to define the value function, and subsequently the optimal tracking controller. Firstly, we define the error function  $l(r) = \|Cr\|_2$  over the relative state space, where  $C$  is a matrix that selects the coordinates of the relative system we wish to penalize so that  $l(r)$  represents the tracking error between the tracking and planning system.

Next, recall that in our relative system, the tracking/planning system controls and disturbances all affect the system. We consider each of these as separate players influencing the outcome of the game. Specifically, it is a pursuit-evasion game [20], where the tracking system takes actions to minimize the maximum tracking error over the remainder of the game, and the planning system and disturbances try to maximize it. Intuitively, this means the tracking and planning systems take the optimal action to pursue/avoid each other respectively.

The strategy the planning system employs is a mapping  $\gamma_p : \mathbb{U}_s(t) \rightarrow \mathbb{U}_p(t)$ , such that the planning system control is chosen based on the tracking control, and restricted to non-anticipative strategies  $\gamma_p \in \Gamma_p(t)$ , defined in [20], [21]. Informally, this means that at each time, the planning system reacts to all tracking system controls so far, but not future controls. The disturbance strategy is similarly defined as  $\gamma_d : \mathbb{U}_s(t) \rightarrow \mathbb{D}(t)$  with  $\gamma_d \in \Gamma_d(t)$ .

We now define the *value function* as the highest cost attained when the tracking system, planning system, and disturbances behave optimally over the interval  $t \in [0, T_{\text{off}}]$ :

$$V(r, t) = \sup_{\gamma_p \in \Gamma_p(t)} \inf_{\gamma_d \in \Gamma_d(t)} \nu(r, t, u_s(\cdot), \gamma_p[u_s](\cdot), \gamma_d[u_s](\cdot)), \quad (5)$$

where  $\nu(r, t, u_s, u_p, d) := \max_{\tau \in [t, T_{\text{off}}]} l(\xi_g(\tau; r, t, u_s, u_p, d))$ . Unlike FaSTrack [16], the relative system (4) corresponding to  $\xi_g$  in (5) is time-varying. Thus, we adopt the forward-time convention from [21] for the value function to avoid confusion when discussing time-varying systems. It is obtained by solving the Hamilton-Jacobi-Isaacs (HJI) variational inequality

$$\begin{aligned} \max \left\{ \frac{\partial V(r, t)}{\partial t} + \min_{u_s \in \mathcal{U}_s} \max_{u_p \in \mathcal{U}_p, d \in \mathcal{D}} \left[ \frac{\partial V(r, t)}{\partial r}^\top g(t, u_s, u_p, d) \right], \right. \\ \left. l(r) - V(r, t) \right\} = 0, \quad (6) \end{aligned}$$

over  $t \in [0, T_{\text{off}}]$  with  $V(r, T_{\text{off}}) = l(r)$ . The solution can be numerically approximated using existing tools in HJ reachability (e.g. the helperOC toolbox [22] or the OptimizedDP library [23]).

The *optimal tracking controller* is obtained as the optimizer of the Hamiltonian as follows:

$$u_s^*(r, t) \in \operatorname{argmin}_{u_s \in \mathcal{U}_s} \max_{u_p \in \mathcal{U}_p, d \in \mathcal{D}} \frac{\partial V(r, t)^\top}{\partial r} g(t, r, u_s, u_p, d). \quad (7)$$

By applying this controller, the trajectory of the relative system is non-increasing in the levels of the value function, as long as the planning system controls and disturbances take values in  $\mathcal{U}_p$  and  $\mathcal{D}$ . To see this, we provide Prop. 1.

**Proposition 1. (Sublevel Set Invariance)** *Let  $s$  and  $p$  be the states of the tracking system (1) and planning system (2) at time  $t \in [0, T_{\text{off}}]$ , and  $r = Ls - Mp$  be the corresponding relative state. For all  $t' \in [t, T_{\text{off}}]$ ,*

$$V(\xi_g^*(t'; r, t), t') \leq V(r, t), \quad (8)$$

where

$$\xi_g^*(t'; r, t) := \xi_g(t'; r, t, u_s^*(\cdot; r, t), u_p^*(\cdot; r, t), d^*(\cdot; r, t)) \quad (9)$$

and  $u_p^*(\cdot; r, t) \in \mathbb{U}_p(t)$ ,  $d^*(\cdot; r, t) \in \mathbb{D}(t)$ ,  $u_s^*(\cdot; r, t) \in \mathbb{U}_s(t)$ ,  $\gamma_p^*[\cdot; r, t] \in \Gamma_p(t)$ ,  $\gamma_d^*[\cdot; r, t] \in \Gamma_d(t)$  are defined as:

$$u_p^*(t'; r, t) = \gamma_p^*[u_s^*(\cdot; r, t); r, t](t'), \quad (10)$$

$$d^*(t'; r, t) = \gamma_d^*[u_s^*(\cdot; r, t); r, t](t'), \quad (11)$$

$$u_s^*(\cdot; r, t) \in \operatorname{arginf}_{u_s(\cdot) \in \mathbb{U}_s(t)} \nu(r, t, u_s(\cdot), \gamma_p^*[u_s; r, t](\cdot), \gamma_d^*[u_s; r, t](\cdot)), \quad (12)$$

$$\gamma_p^*[\cdot; r, t] \in \operatorname{argsup}_{\gamma_p \in \Gamma_p(t)} \inf_{u_s(\cdot) \in \mathbb{U}_s(t)} \nu(r, t, u_s(\cdot), \gamma_p[u_s](\cdot), \gamma_d^*[u_s; r, t](\cdot)),$$

$$\gamma_d^*[\cdot; r, t] \in \operatorname{argsup}_{\gamma_d \in \Gamma_d(t)} \sup_{\gamma_p \in \Gamma_p(t)} \inf_{u_s(\cdot) \in \mathbb{U}_s(t)} \nu(r, t, u_s(\cdot), \gamma_p[u_s](\cdot), \gamma_d[u_s](\cdot)).$$

Here,  $\gamma_p^*[\cdot; r, t]$  and  $\gamma_d^*[\cdot; r, t]$  denote the worst-case non-anticipative strategies for the planning system controls and disturbance respectively,  $u_s^*(\cdot; r, t)$  denotes the *optimal tracking system controls* assuming that the planning system controls and disturbances are chosen according to those worst-case strategies, and  $u_p^*(\cdot; r, t)$  and  $d^*(\cdot; r, t)$  denote the corresponding *worst-case planning system controls and disturbances*. Moreover,  $\xi_g^*(\cdot; r, t) : [t, T_{\text{off}}] \rightarrow \mathcal{R}$  denotes the *optimal relative system trajectory* when the optimal tracking system controls and worst-case planning system controls and disturbance are applied to the relative system (3). These worst-case considerations are usually conservative since the planning system is typically not always adversarially escaping the tracking system. However, they guarantee the optimal tracking controller (7) achieves sublevel set invariance in  $V(r, t)$  (see Prop. 1) with arbitrary planning system controls, which we elaborate upon in Remark 1. This decouples the two so the former can be pre-computed offline and the latter chosen online, which is key to enabling real-time performance since solving (6) for  $V(r, t)$  is computationally expensive. Although not our main focus, existing works (see [17], [24]) reduce the conservatism associated with worst-case planning systems. This issue is somewhat orthogonal to our results since their techniques can be applied on top of what we propose.

**Remark 1.** *Although Prop. 1 requires  $u_s^*$  (12) for the tracking system controls and  $u_p^*$  (10),  $d^*$  (11) for the planning system controls and the disturbances, sublevel set invariance should still hold when the optimal tracking controller (7) is applied and*



the planning system controls and disturbances arbitrarily take values in  $\mathcal{U}_p$  and  $\mathcal{D}$ . There are two reasons for this. Firstly, the controls generated by the optimal tracking controller (7) with the worst-case disturbances and planning system controls should be equivalent to (12). This is true for single-player Bolza problems with differentiable value functions [25], but should also usually hold in our two-player minimum cost problem setting with an almost everywhere differentiable value function. Secondly, compared to the worst-case planning system controls  $u_p^*$  and disturbances  $d^*$ , it should be “easier” for the optimal tracking controller (7) to oppose arbitrary planning system controls and disturbances taking values in  $\mathcal{U}_p$  and  $\mathcal{D}$  and hence achieve sublevel set invariance.

**Remark 2.** The guarantee in Prop. 1 requires that the optimal tracking controller  $u_s^*$  is exactly obtained, which relies on exactly obtaining  $V$ . In practice, this may not be achieved when numerically solving (6) over a grid (although the numerical PDE solution converges to the exact solution as the grid spacing goes to zero). In this paper, we assume that  $V$  is exactly obtained when providing guarantees, and leave consideration of the effects of errors to future work.

The overall steps required for the offline computation are summarized in Alg. 1. The most computationally expensive procedure is line 3 since dynamic programming suffers from the curse of dimensionality. However, recent techniques have shown promise for solving higher dimensional HJ PDEs, such as the decomposition of systems into smaller subsystems [26], and the implementation of high-performance solvers [23] — which was used for the examples in this work — although this remains an active field of research. After successfully computing  $V(r, t)$  and  $u_s^*(r, t)$ , one can apply them online in real time, which will be useful in Sec. V and VI.

#### Algorithm 1 OfflineStage

- 1: **Input:**  $L \in \mathbb{R}^{n_r \times n_s}$ ,  $M \in \mathbb{R}^{n_r \times n_p}$ ,  $C$ ,  $f(t, s, u_s, d)$  for  $t \in [0, T_{\text{off}}]$ ,  $h(p, u_p)$ ,  $T_{\text{off}} \in [0, T]$
- 2: Formulate  $g(t, r, u_s, u_p, d)$ ,  $t \in [0, T_{\text{off}}]$  using (3)
- 3: Compute  $V(r, t)$ ,  $t \in [0, T_{\text{off}}]$  and  $\frac{\partial}{\partial r} V(r, t)$ ,  $t \in [0, T_{\text{off}}]$  using dynamic programming in Sec. III-B
- 4: Construct the optimal safe controller  $u_s^*(r, t)$  from (7)
- 5: **Output:**  $V(r, t)$ ,  $u_s^*(r, t)$  for  $t \in [0, T_{\text{off}}]$

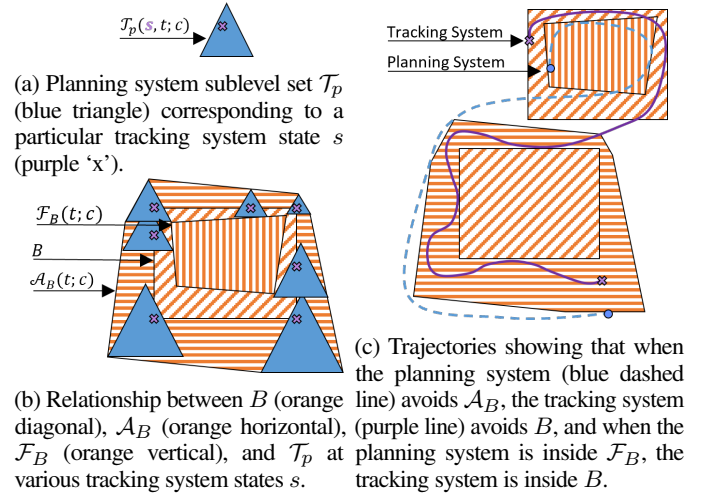
## IV. TRAJECTORY PLANNING FOR TIME-VARYING SYSTEMS

In this section, we describe the online planning component of our method. We first introduce essential mathematical objects in Sec. IV-A, before describing our algorithm in Sec. IV-B. Subsequently, in Sec. IV-C, we theoretically analyze the properties of our planning method with respect to our objectives of constraint satisfaction and goal-reaching when combined with the optimal tracking controller (7). Throughout this section, we assume that the state constraints  $\mathcal{C} \subseteq \mathcal{S}$  are static (i.e. they do not change over time).

### A. The Planning System

#### Sublevel Set, Set Satisfaction Map, and Set Avoidance Map

Recall that the key idea from Prop. 1 is that the optimal relative system trajectory (9) is non-increasing in the levels of  $V$  over time (8). Given the tracking system state  $s$  at time  $t$ , we can find the set



**Fig. 1:** Illustrations showing how  $\mathcal{T}_p$  is used to construct  $\mathcal{F}_B$  and  $\mathcal{A}_B$ , and their importance for the planning system and tracking system trajectories, when  $n_s = n_p$ .

of planning system states  $p$  such that the relative system is within the sublevel set corresponding to a particular value level  $c \in \mathbb{R}$  of  $V$ . This is called the *planning system sublevel set*  $\mathcal{T}_p$ , and is defined as

$$\mathcal{T}_p(s, t; c) := \{p : V(Ls - Mp, t) \leq c\} \subseteq \mathcal{P}, \quad (13)$$

where  $t \in [0, T_{\text{off}}]$ . For any given  $s$  and  $t$ , the set  $\mathcal{T}_p(s, t; c)$  is nonempty if  $c \geq \underline{V}(s, t)$ , where  $\underline{V} : \mathcal{S} \times [0, T_{\text{off}}] \rightarrow \mathbb{R}$  denotes the *minimum value level function*:

$$\underline{V}(s, t) := \min_p V(Ls - Mp, t). \quad (14)$$

Next, let us define the *set satisfaction map* as

$$\mathcal{F}_B(t; c) := \bigcap_{s \in B^c} \mathcal{T}_p^c(s, t; c). \quad (15)$$

given set  $B \subseteq \mathcal{S}$  and  $t \in [0, T_{\text{off}}]$  (visualized in Fig. 1b), where  $c$  denotes set complement. It represents the set of planning system states that guarantee the tracking system state is inside the set  $B$ , supposing the relative system is in the sublevel set of  $V$  corresponding to  $c$ . It is useful for translating constraints and goal requirements in the tracking system space  $\mathcal{S}$  to the planning system space  $\mathcal{P}$ . This can be seen in Fig. 1c for the case where  $n_s = n_p$ , such that when the planning system is inside  $\mathcal{F}_B$ , the tracking system is guaranteed to be inside set  $B$ . Note that  $n_s = n_p$  does not need to hold in general.

A related object is the *set avoidance map*, defined as  $\mathcal{A}_B(t; c) := \bigcup_{s \in B} \mathcal{T}_p(s, t; c)$  (visualized in Fig. 1b). It is the set of planning system states which, when avoided by the planning system, ensures the tracking system avoids  $B$ , shown in Fig. 1c. This map is useful when it is more natural to think of constraint satisfaction as avoiding an undesirable set  $B$  (e.g. obstacles). It can be easily seen that  $p \notin \mathcal{A}_B(t; c)$  is equivalent to  $p \in \mathcal{F}_{B^c}(t; c)$ , ensuring the tracking system is in  $B^c$ .

### B. Trajectory Planning Method Over a Bounded Interval

We describe our trajectory planning method for time-varying systems over any bounded time interval  $[t_i, t_f] \subseteq [0, T_{\text{off}}]$ , which is summarized in Alg. 2. The inputs to the algorithm include the value function  $V(r, t)$ ,  $t \in [0, T_{\text{off}}]$  — which is obtained from OfflineStage

(Alg. 1) — the planning system model (2), the goal  $\mathcal{G} \subseteq \mathcal{S}$ , the planning start time  $t_i \in [0, T_{\text{off}}]$ , and the planning end time  $t_f \in [t_i, T_{\text{off}}]$ . The static constraints  $\mathcal{C} \subseteq \mathcal{S}$  are also supplied as an input. Moreover, the initial planning state is an input with  $p \in \mathcal{T}_p(s, t_i; c)$  enforced, since it is required for establishing safety guarantees. Its importance will be discussed in Sec. IV-C. The final input is a chosen value level  $c$  satisfying  $c \geq \underline{V}(s, t)$ , where  $s \in \mathcal{S}$  is the tracking system state at time  $t_i$ . Although enforcing  $c = \underline{V}(s, t)$  is sufficient for  $\mathcal{T}_p(s, t; c) \neq \emptyset$ , allowing  $c$  to be flexibly chosen enables the adaptive performance of the controlled system. This will become apparent in Sec. V.

Firstly, using  $\mathcal{F}_B(t; c)$  (15), we compute the *planning system constraints*  $\mathcal{C}_p$  from  $\mathcal{C}$ , and the *planning system goal*  $\mathcal{G}_p$  from  $\mathcal{G}$ , over the interval  $[t_i, t_f]$  for the particular level  $c$  we have chosen. Following this, we plan a trajectory  $\hat{\xi}_h(t)$  over the interval  $t \in [t_i, t_f]$ , where  $\hat{u}_p(\cdot) \in \mathcal{U}_p(t_i)$  are the controls for the planned trajectory. The planned trajectory must satisfy Conditions C1-C3 and optionally C4.

---

**Algorithm 2** PlanTrajectoryOverInterval

---

- 1: **Input:**  $V(r, t)$  for  $t \in [0, T_{\text{off}}]$ ,  $h(p, u_p)$  for  $t_i \in [0, T_{\text{off}}]$ ,  $s \in \mathcal{S}$ ,  $t_f \in [t_i, T_{\text{off}}]$ ,  $\mathcal{C} \subseteq \mathcal{S}$ ,  $\mathcal{G} \subseteq \mathcal{S}$ ,  $c$  satisfying  $c \geq \underline{V}(s, t)$ ,  $p$  satisfying  $p \in \mathcal{T}_p(s, t_i; c)$
  - 2: Compute  $\mathcal{C}_p(t) \leftarrow \mathcal{F}_C(t; c)$  for  $t \in [t_i, t_f]$  from (15)
  - 3: Compute  $\mathcal{G}_p(t) \leftarrow \mathcal{F}_G(t; c)$  for  $t \in [t_i, t_f]$  from (15)
  - 4: Plan trajectory  $\hat{\xi}_h(t)$ ,  $t \in [t_i, t_f]$  based on  $h$  satisfying C1-C3, and optionally C4 (see Remark 3)
  - 5: **Output:**  $\hat{\xi}_h(t)$  for  $t \in [t_i, t_f]$
- 

- C1.  $\hat{u}_p(\cdot) \in \mathcal{U}_p(t_i)$  (Planning system control constraint satisfaction)
- C2.  $\hat{\xi}_h(t) = \xi_h(t; p, t, \hat{u}_p(\cdot))$  (Planning system dynamics satisfaction)
- C3. For all  $t \in [t_i, t_f]$ ,  $\hat{\xi}_h(t) \in \mathcal{C}_p(t)$  (Planning system state constraint satisfaction)
- C4. There exists  $T_r \in [t_i, t_f]$  such that  $\hat{\xi}_h(T_r) \in \mathcal{G}_p(t)$  (Planning system goal reaching) (optional)

**Remark 3.** We consider C4 a soft constraint in practice, since requiring there is some time  $T_r$  so that the planning system is guaranteed to hit the planning system goal region is restrictive when the goal  $\mathcal{G}_p$  is far from the system and the interval  $[t_i, t_f]$  is short. However, it is important for connecting Alg. 3 to the analysis in Sec. IV-C.

**Remark 4.** Our framework supports arbitrary methods for generating the planned trajectory in line 5, as long as it can satisfy C1-C4. One possibility is MPC, which will be demonstrated in Sec. VII-C.

The trajectory  $\hat{\xi}_h(t)$ ,  $t \in [t_i, t_f]$  is then returned as the output from the planning procedure. Note that for static constraints, the optimal tracking controller (7) can already be applied to follow the planned trajectory to achieve constraint satisfaction and goal-reaching.

### C. Theoretical Guarantees for Tracking Planned Trajectories

In this section, we theoretically justify combining the optimal tracking controller with the trajectory planning method in Alg. 2 by providing Thm. 3, which forms the basis of our analysis for trajectory replanning in later sections. Before doing so, let us define the *optimal tracking system trajectory*  $\xi_f^* : [t, T_{\text{off}}] \rightarrow \mathcal{S}$  and the *worst-case planning system trajectory*  $\xi_h^* : [t, T_{\text{off}}] \rightarrow \mathcal{P}$  in (16) and

(17) respectively:

$$\xi_f^*(t'; s, p, t) := \xi_f(t'; s, t, u_s^*(\cdot; Ls - Mp, t)), \quad (16)$$

$$d^*(\cdot; Ls - Mp, t),$$

$$\xi_h^*(t'; p, s, t) := \xi_h(t'; p, t, u_p^*(\cdot; Ls - Mp, t)), \quad (17)$$

They correspond to the tracking system (1) and planning system (2) trajectories starting from states  $s$  and  $p$  respectively, under the optimal tracking system controls (12), and worst-case planning system controls (10) and disturbances (11).

We now provide Lem. 2. It says that if the value level  $c$  is sufficiently large and the planning system is initialized in the planning system sublevel set  $\mathcal{T}_p$ , then if the worst-case planning system trajectory (17) is in  $\mathcal{F}_B$ , the optimal tracking system trajectory is in  $B$ .

**Lemma 2.** Consider  $s \in \mathcal{S}$ ,  $p \in \mathcal{P}$ ,  $t \in [0, T]$ ,  $c \in \mathbb{R}$  and  $B \subseteq \mathcal{S}$ . Suppose  $c \geq \underline{V}(s, t)$ , and  $p \in \mathcal{T}_p(s, t; c)$ . Then, we have

$$\xi_h^*(t'; p, t) \in \mathcal{F}_B(t'; c) \implies \xi_f^*(t'; s, t) \in B,$$

for all  $t' \in [t, T]$ .

Thm. 3 follows as an application of Lem. 2 and says that under the same conditions as Lem. 2, if the worst-case planning system trajectory satisfies the planning system constraints  $\mathcal{F}_C(t; c)$ , the optimal tracking system trajectory achieves constraint satisfaction in  $\mathcal{C}$ . Moreover, if the worst-case planning system trajectory satisfies the planning system goal  $\mathcal{F}_G(t; c)$ , the optimal tracking system trajectory is in the goal  $\mathcal{G}$ .

**Theorem 3.** Consider tracking system state  $s \in \mathcal{S}$  and planning system state  $p \in \mathcal{P}$  at planning start time  $t_i \in [0, T_{\text{off}}]$ . Let  $t_f \in [t_i, T_{\text{off}}]$  be the planning end time, and let  $c \in \mathbb{R}$  denote the value level. Suppose  $c$  and  $p$  satisfy

$$c \geq \underline{V}(s, t_i) \quad \text{and} \quad p \in \mathcal{T}_p(s, t_i; c). \quad (18)$$

Then for any  $t \in [t_i, t_f]$ , the following hold:

$$\xi_h^*(t; p, t_i) \in \mathcal{F}_C(t; c) \implies \xi_f^*(t; p, t_i) \in \mathcal{C}, \quad (19)$$

$$\xi_h^*(t; p, t_i) \in \mathcal{F}_G(t; c) \implies \xi_f^*(t; s, t_i) \in \mathcal{G}. \quad (20)$$

**Remark 5.** Thm. 3 provides sufficient conditions for constraint satisfaction and goal reaching for the tracking system (1) over  $[t_i, t_f]$ , supposing that the optimal tracking system controls  $u_s^*$  (12) and worst-case disturbances  $d^*$  (11) are applied to the tracking system (1), and the worst-case planning system controls  $u_p^*$  (10) are applied to the planning system (2). Since Thm. 3 relies on Prop. 1, the result should still hold when these are replaced by the optimal tracking controller (7), and arbitrary disturbances and planning system controls in  $\mathcal{D}$  and  $\mathcal{U}_p$ , for the same reasons pointed out in Remark 1.

We now discuss the connections between Thm. 3 and the scheme where the optimal tracking controller (7) is applied to track trajectories planned via Alg. 2. Condition C2 ensures that the planning system evolves according to (2), and C1 ensures the planning system controls take values in  $\mathcal{U}_p$ , so supposing that the disturbances take values in  $\mathcal{D}$ , the guarantees in Thm. 3 are still applicable to the tracking system controlled by the proposed scheme via Remark 5 — we only need to check the conditions in Thm. 3 hold after replacing the worst-case planned trajectory  $\xi_h^*$  with  $\hat{\xi}_h$ . In Alg. 2, the value level  $c$  and the planning system state  $p$  satisfy (18), and the planned trajectory  $\hat{\xi}_h$  satisfies C3 and (optionally) C4. Therefore,

(19) and (20) can be applied to conclude that constraint satisfaction and (optionally) goal-reaching are achieved over the interval  $[t_i, t_f]$ .

Although safety is guaranteed when  $\mathcal{C}$  is static, the current procedure does not handle constraints updated online. This limitation motivates trajectory replanning in Sec. V.

## V. TRAJECTORY REPLANNING AND TRACKING

Although trajectory planning using Alg. 2 from Sec. IV can be combined with tracking using (7) to achieve static constraint satisfaction and goal-reaching, our problem setup in II involves constraints that are updated online in real time. We clarify exactly what we mean by online constraint updates in Sec. V-A. Subsequently, we propose a method that handles online constraint updates by replanning trajectories, over a task horizon  $T_{\text{run}} \leq T_{\text{off}}$  in Sec. V-B. Moreover, our method allows for flexible reinitialization of the planning system, which can be exploited for improved performance. The theoretical properties of our method are subsequently analyzed in Sec. V-C with respect to our objectives of online constraint satisfaction and goal-reaching.

### A. The Online Constraint Satisfaction Problem

Throughout this section, we consider constraints that are updated online in real time, denoted as  $\mathcal{C}(t) \subseteq \mathcal{S}$  for  $t \in [0, T_{\text{run}}]$ . Since  $\mathcal{C}(t)$  represents online constraints, at time  $t \in [0, T_{\text{run}}]$ ,  $\mathcal{C}(t')$  is unknown for  $t' > t$ . Moreover, it is assumed that  $\mathcal{C}(t)$  changes at a finite number of increasing time instances  $\{t_i^{\text{update}}\}_{i=0}^{N_{\text{update}}} \subseteq [0, T_{\text{run}}]$  with  $t_0^{\text{update}} = 0$  and  $t_{N_{\text{update}}}^{\text{update}} = T_{\text{run}}$ , where  $N_{\text{update}}$  is the number of times the constraints are updated. Specifically, for all  $i \in \{0, \dots, N_{\text{update}} - 1\}$  and  $t_1, t_2 \in [t_i^{\text{update}}, t_{i+1}^{\text{update}})$ , we assume that

$$\mathcal{C}(t_1) = \mathcal{C}(t_2). \quad (21)$$

### B. Method for Replanning and Tracking

Our online trajectory replanning and tracking method for time-varying systems subject to disturbances  $d \in \mathcal{D}$  over a bounded time interval  $t \in [0, T_{\text{run}}]$  is summarised in Alg. 3.

The inputs to the procedure include the matrices  $L \in \mathbb{R}^{n_r \times n_s}$  and  $M \in \mathbb{R}^{n_r \times n_p}$  required for formulating the relative model (3), the tracking system model (1), the planning system model (2), the goal  $\mathcal{G} \subseteq \mathcal{S}$  and the task horizon  $T_{\text{run}}$ . Firstly, during the offline stage of the algorithm, we compute the value function  $V(r, t)$  and the optimal tracking controller  $u_s^*(r, t)$  over  $t \in [0, T_{\text{off}}]$  via OfflineStage (Alg. 1). Subsequent lines are then implemented online in real time. In particular, for each time over the task horizon  $t \in [0, T_{\text{off}}]$ , the current state  $s$  of the tracking system is measured, and if  $s$  is inside  $\mathcal{G}$ , then the algorithm stops running. The current state constraint set considered within the algorithm —  $\mathcal{C}_{\text{curr}}$  — is selected based on the most recent measurements of the updated constraints  $\mathcal{C}(t)$ . If  $\mathcal{C}_{\text{curr}}$  has changed or we choose to replan at the current time  $t$ , then  $k$  is incremented, we store the current time and tracking system state as  $t_k$  and  $s_k$  respectively, choose a value level  $c_k$  satisfying  $c_k \geq \underline{V}(s_k, t_k)$ , choose an initial planning system state  $p_k$  satisfying  $p_k \in \mathcal{T}_p(s_k, t_k; c_k)$ , and then apply the trajectory replanning procedure *PlanOverBoundedInterval* from Alg. 2 to solve for a trajectory  $\hat{\xi}_{h,k}(t')$  over  $t' \in [t_k, T_{\text{run}}]$ . Outside of the trajectory replanning procedure, the current state of the planning system  $\hat{\xi}_h^R(t)$  is set equal to  $\hat{\xi}_{h,k}(t)$ , and the optimal tracking controller (7) is applied to the

### Algorithm 3 Safe Trajectory Replanning and Tracking for Time-Varying Systems

```

1: Input:  $L \in \mathbb{R}^{n_r \times n_s}$ ,  $M \in \mathbb{R}^{n_r \times n_p}$ ,  $T_{\text{off}} \leq T$ ,
    $f(t, s, u_s, d)$  for  $t \in [0, T_{\text{off}}]$ ,  $h(p, d)$ ,  $\mathcal{G} \subseteq \mathcal{S}$ ,  $T_{\text{run}} \leq T_{\text{off}}$ 
2: Compute  $V(r, t)$ ,  $t \in [0, T_{\text{off}}]$  and  $u_s^*(r, t)$ ,  $t \in [0, T_{\text{off}}]$  using
   OfflineStage( $L, M, \mathcal{C}, f, h$ ) from Alg. 1
3:  $k \leftarrow -1$ 
4: for  $t \in [0, T_{\text{run}}]$  do
5:   Measure current tracking system state  $s$  from (1)
6:   if  $s \in \mathcal{G}$  then
7:     Return (goal reached).
8:   end if
9:   Update constraints  $\mathcal{C}_{\text{curr}} \leftarrow \mathcal{C}(t)$ 
10:  if ( $\mathcal{C}_{\text{curr}}$  has changed) OR (decide to replan at time  $t$ 
    (see Remark 6)) then
11:     $k \leftarrow k + 1$ 
12:     $(t_k, s_k) \leftarrow (t, s)$ 
13:    Choose  $c_k \geq \underline{V}(s_k, t_k)$  from (14) (see Remark 7)
14:    Choose  $p_k \in \mathcal{T}_p(s_k, t_k; c_k)$  from (13) (See Remark
    7)
15:     $\hat{\xi}_{h,k}(t')$ ,  $t' \in [t_k, T_{\text{run}}] \leftarrow \text{PlanTrajectoryOverInterval}$ 
    ( $V, h, t_k, s_k, T_{\text{run}}, \mathcal{C}_{\text{curr}}, \mathcal{G}, c_k, p_k$ ) from Alg. 2
16:  end if
17:   $\hat{\xi}_h^R(t) \leftarrow \hat{\xi}_{h,k}(t)$ 
18:  Compute  $u_s \leftarrow u_s^*(Ls - M\hat{\xi}_h^R(t), t)$  following (7)
19:  Apply  $u_s$  as control input to tracking system (1)
20: end for

```

tracking system (1) based on the current relative state  $Ls_k - M\hat{\xi}_h^R(t)$  and current time  $t$ . The online procedure repeats until  $T_{\text{run}}$ . The online phase of Alg. 3 is visualized as a block diagram in Fig. 2.

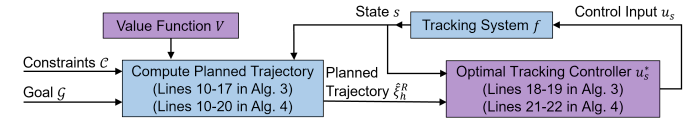


Fig. 2: Block diagram for online phase of Algs. 3 and 4. Objects computed offline are colored purple.

The most expensive part of Alg. 3 is line 2 when OfflineStage is called. The online stage (lines 4–20) can be executed in real time, as long as a computationally efficient real-time planning procedure is selected in line 15 (recall Remark 4). This is possible by taking advantage of the framework's support for simple planning models.

Note that an alternative strategy for choosing controls in line 18 is possible, where an arbitrary controller is used when the relative system  $Ls - M\hat{\xi}_h^R(t)$  is within the interior of the sublevel set of  $V(r, t)$  corresponding to  $c_k$ , and (7) is applied when the relative system is close to the border. We refer the reader to [16] for further details.

**Remark 6.** Without loss of generality, trajectory planning must always be enforced at time  $t = 0$  and when constraints are updated. Beyond this, our method can flexibly support multiple different conditions in line 10. For example, replanning may be performed after every time interval of some fixed length, as is common in receding horizon control.



**Remark 7.** We do not prescribe a fixed way to set  $c_k$  and  $p_k$  in Alg. 3, except they must satisfy  $c_k \geq \underline{V}(s_k, t_k)$  and  $p_k \in \mathcal{T}_p(s_k, t_k; c_k)$ . This is unlike FaSTrack [16], which reinitializes the planning system state as equal to the state at time  $t$  from the previous planned trajectory in line 15. In other words, our planning system can “teleport”, which can be exploited for adaptive performance in the constraint satisfaction and goal-reaching problem. Specifically, choosing larger  $c_k$  allows  $p_k$  to be initialized further away from  $s_k$ , which encourages the optimal tracking controller (7) to apply controls to catch up. On the other hand, smaller  $c_k$  will produce less restrictive planning system constraints  $\mathcal{C}_p(t)$ , which may be required for navigation through tight environments. This tradeoff will be demonstrated inside simulations in Sec. VII-D.

### C. Theoretical Guarantees for Replanning and Tracking

In this section, we theoretically justify the usage of Alg. 3 to compute controls for the tracking system (1) via Cor. 4. Before doing so, we introduce mathematical objects required for our analysis.

Let  $K \in \mathbb{N}$  denote the number of times trajectory replanning occurs, and suppose  $T_{\text{run}} \leq T_{\text{off}}$ . Consider a sequence of planning system states  $\{p_k\}_{k=0}^{K-1} \subseteq \mathcal{P}$ , and a sequence of increasing time instances  $\{t_k\}_{k=0}^K \subseteq [0, T_{\text{off}}]$  where replanning occurs, satisfying  $t_0 = 0$  and  $t_K = T_{\text{run}}$ . Assume  $\{t_i^{\text{update}}\}_{i=0}^{N_{\text{update}}} \subseteq \{t_k\}_{k=0}^K$  holds, so replanning occurs whenever constraints are updated. We define the replanned trajectory  $\xi_h^R : [0, T_{\text{run}}) \rightarrow \mathcal{P}$  as the concatenation of the worst-case planning system trajectories (17) on each sub-interval  $[t_k, t_{k+1})$  for  $0 \leq k \leq K-1$ , where each trajectory is determined by the planning system state  $p_k$  and tracking system state  $s_k$  at time  $t_k$ :

$$\xi_h^R(t) := \xi_h^*(t; p_k, s_k, t_k) \quad t \in [t_k, t_{k+1}), \quad (22)$$

for  $0 \leq k \leq K-1$ . Here,  $s_k$  is recursively defined based on the optimal tracking system trajectory (16) initialized at tracking system state  $s_{k-1}$  and  $p_{k-1}$  at time  $t_{k-1}$ :

$$s_k := \xi_f^*(t_k; s_{k-1}, p_{k-1}, t_{k-1}),$$

and  $s_0$  is the state of the tracking system (1) at time 0.

Next, define the replanned tracking system controls  $u_s^R : [0, T_{\text{run}}) \rightarrow \mathcal{U}_s$  as the concatenation of the optimal tracking system controls (12) over each sub-interval between replanning time instances:

$$u_s^R(t) := u_s^*(t; Ls_k - Mp_k, t_k) \quad t \in [t_k, t_{k+1}), \quad (23)$$

for  $0 \leq k \leq K-1$ . Moreover, define the replanned disturbances  $d^R : [0, T_{\text{run}}) \rightarrow \mathcal{D}$  as the concatenation of the worst-case disturbances (11) over each sub-interval:

$$d^R(t) := d^*(t; Ls_k - Mp_k, t_k) \quad t \in [t_k, t_{k+1}). \quad (24)$$

for  $0 \leq k \leq K-1$ . Next, the replanned tracking system trajectory is the trajectory of the tracking system starting from  $s_0$  at time 0 under the replanned tracking system controls (23) and disturbances (24):

$$\xi_f^R(t) := \xi_f(t; s_0, 0, u_s^R(\cdot), d^R(\cdot)), t \in [0, T_{\text{run}}).$$

By definition,  $\xi_h^R(t)$ ,  $u_s^R(t)$ ,  $d^R(t)$  over  $t \in [0, T_{\text{off}})$  are uniquely determined by  $s_0$ ,  $\{p_k\}_{k=0}^{K-1}$  and  $\{t_k\}_{k=0}^K$ , where  $k'(t) = \max\{k \in \mathbb{N}_0 : t_k \leq t\}$ .

With these mathematical objects defined, we are now ready to provide Cor. 4. It says that for each replanning step  $k \in \{0, \dots, K-1\}$

and associated time interval  $[t_k, t_{k+1})$ , as long as the corresponding value level  $c_k$  is sufficiently large and the initial planning system state  $p_k$  on this interval is inside the planning system sublevel set  $\mathcal{T}_p(\xi_f^R(t_k), t_k; c_k)$  (25), then at any time in this interval that the replanned trajectory  $\xi_h^R(t)$  satisfies the planning system constraints  $\mathcal{F}_{\mathcal{C}(t)}(t; c_k)$  (goal  $\mathcal{F}_{\mathcal{G}}(t; c_k)$ ), the replanned tracking system trajectory  $\xi_f^R(t)$  satisfies the constraints  $\mathcal{C}(t)$  (26) (goal  $\mathcal{G}$  (27)).

**Corollary 4.** Let  $\{c_k\}_{k=0}^{K-1} \subseteq \mathbb{R}$  be a sequence of value levels. Suppose for all  $k \in \{0, \dots, K-1\}$ ,

$$c_k \geq \underline{V}(\xi_f^R(t_k), t_k) \quad \text{and} \quad p_k \in \mathcal{T}_p(\xi_f^R(t_k), t_k; c_k). \quad (25)$$

Then, for any  $k \in \{0, \dots, K-1\}$  and any  $t \in [t_k, t_{k+1})$ , we have:

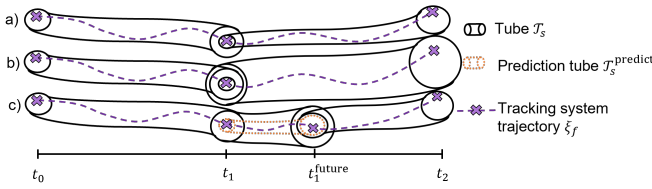
$$\xi_h^R(t) \in \mathcal{F}_{\mathcal{C}(t_k)}(t; c_k) \implies \xi_f^R(t) \in \mathcal{C}(t), \quad (26)$$

$$\xi_h^R(t) \in \mathcal{G}_p(t; c_k) \implies \xi_f^R(t) \in \mathcal{G}. \quad (27)$$

**Remark 8.** Cor. 4 provides sufficient conditions for collision avoidance and/or goal-reaching over the task horizon  $[0, T_{\text{run}})$  supposing that within each sub-interval  $[t_k, t_{k+1})$  for  $k \in \{0, \dots, K-1\}$ , the optimal tracking system controls  $u_s^*$  (12) and worst-case disturbances (11) are applied to (1) and the worst-case planning system controls (10) are applied to (2), after reinitializing the planning system state at  $p_k$ . The proof naturally follows by applying Thm. 3 in each sub-interval of the task horizon, and therefore the guarantees should also hold when the optimal tracking controller (7) and arbitrary planning system controls and disturbances taking values in  $\mathcal{U}_p$  and  $\mathcal{D}$  are applied within each sub-interval, for the same reasons pointed out in Remark 5.

We now discuss the connections between Alg. 3 and Cor. 4. Note that every time replanning occurs in Alg. 3: 1) the planning system controls for each replanned trajectory take values in  $\mathcal{U}_p$  since C1 must be satisfied when planning the trajectory in line 15; 2) the tracking system controls are generated by the optimal tracking controller (7) in line 18; 3) the disturbances take values in  $\mathcal{D}$  by assumption. Thus, the results in Cor. 4 apply to the tracking system controlled by Alg. 3, as discussed in Remark 8. To see that collision avoidance or goal-reaching is achieved, we only need to check that the conditions in Cor. 4 are satisfied after replacing  $\xi_h^R(t)$  and  $\xi_f^R(t_k)$  with  $\hat{\xi}_h^R(t)$  and  $s_k$  from Alg. 3. This can be verified by comparing lines 13 and 14 in Alg. 3 to (25), and  $\hat{\xi}_f^R(t) = \hat{\xi}_{h,k}^R(t)$  (which satisfies C3 and C4 via line 15) to (26) and (27). Therefore, (26) and (optionally) (27) can be applied to find that constraint satisfaction and goal-reaching are achieved within the corresponding sub-interval up to the next time  $t_{k+1}$  that replanning occurs. Since replanning is repeated throughout the task horizon, we can conclude that Alg. 3 ensures the tracking system (1) achieves constraint satisfaction and (optionally) goal reaching over the entire task horizon.

**Remark 9.** The argument in Cor. 4 is similar to the idea of sequentially composing tubes (or funnels) in [4], but there are some differences. Recall that a tube is a family of subsets of the state space  $\mathcal{S}$  parameterized over time  $t$  which the tracking system is guaranteed to remain inside. For each sub-interval  $[t_k, t_{k+1})$  where  $0 \leq k \leq K-1$ , there is an implicit tube  $\mathcal{T}_s(t; \xi_h^R(\cdot), c_k) := \{s \in \mathcal{S} : V(Ls - M\xi_h^R(t), t) \leq c_k\}$  associated with the planned trajectory  $\xi_h^R$  and value level  $c_k$ . Condition (25) ensures that the ‘inlet’ of each tube contains the specific tracking system at



**Fig. 3:** Illustration of tubes containing the tracking system trajectory. Case a) represents the method in this paper where the inlet just needs to contain the tracking system state. Case b) is analogous to [4] where the outlet of the previous tube is a subset of the inlet of the current tube. Case c) represents a modification to handle non-instantaneous trajectory planning.

the corresponding replanning time, as illustrated in Fig. 3 a). This is unlike [4], which requires that the ‘outlet’ of the previous tube is a subset of the ‘inlet’ of the current tube, as illustrated in Fig. 3 b).

**Remark 10.** Our strategy and analysis do not account for the time required to run the online part of Alg. 3, which is subject to hardware-related constraints. Essentially, we assume all lines in Alg. 3 resolve instantaneously, which is not true, even when real-time planning methods exploiting simple planning models are chosen. A more realistic assumption is the online computation time is concentrated in line 15 of Alg. 3 (during planning), since it dominates computation in practice. Although computation time can be drastically reduced using simple planning system models (e.g. single integrator), it would still not be instantaneous. However, assuming we know an upper bound  $T_{\text{compute}} \geq 0$  on the computation time for planning, we can modify Alg. 3 to account for it. This is done by turning line 15 into a parallel process that generates a new planned trajectory from a future time  $t_k^{\text{future}}$  greater than the current time  $t_k$  plus  $T_{\text{compute}}$ , while simultaneously tracking the previously planned trajectory with  $u_s^*$ . Note that the new planned trajectory cannot be initialized based on the ‘exact’ tracking system state at  $t_k^{\text{future}}$  analogously to line 14 since this state is unknown. However, we can instead define a ‘prediction tube’  $\mathcal{T}_s^{\text{predict}}(t; s_k, \xi_h^R(\cdot)) := \mathcal{T}_s(t; \xi_h^R(\cdot), V(Ls_k - M\xi_h^R(t_k), t_k))$  guaranteed to contain the tracking system over  $t \in [t_k, t_k^{\text{future}}]$ , and ensure that the inlet of the next tube contains its outlet, as visualized in Fig. 3 c). This is implemented by initializing the next planned trajectory at  $t_k^{\text{future}}$  in the set  $\bigcap_{s \in \mathcal{T}_s^{\text{predict}}(t_k^{\text{future}}; s_k, \xi_h^R(\cdot))} \mathcal{T}_p(s, t_k^{\text{future}}, c_k)$  with  $c_k \geq c$  and tracking the trajectory with  $u_s^*$  after  $t_k^{\text{future}}$ . Despite outlining the modifications to account for  $T_{\text{compute}}$ , we continue assuming instantaneous planning in the paper for simplicity.

## VI. TRAJECTORY

### REPLANNING AND TRACKING FOR PERIODIC SYSTEMS

The trajectory replanning and tracking method in Sec. V can guarantee collision avoidance and goal-reaching with constraints updated online in real time but is only applicable over a restricted horizon  $T_{\text{run}} \leq T_{\text{off}}$ . We overcome this limitation for the specific case of periodic systems (Sec. VI-A), allowing for  $T_{\text{run}} \geq 0$  generally. We propose a trajectory replanning and tracking method for periodic systems in Sec. VI-B. The theoretical properties of our method are subsequently analyzed in Sec. VI-C with respect to our objectives of online constraint satisfaction and goal-reaching.

### A. Periodic Tracking Systems

Throughout this section, assume that the tracking system dynamics are periodic:

$$f(t, s, u_s, d) = f(t + \tau, s, u_s, d), \quad t \in [0, \infty), \quad (28)$$

where  $u_s \in \mathcal{U}_s$ ,  $u_p \in \mathcal{U}_p$ ,  $d \in \mathcal{D}$ , and  $\tau \in [0, \infty)$  is the period. Although this seems like a strong assumption, there are practical settings where it is reasonable, such as underwater robots subject to wave disturbances [27]. Moreover, we assume  $T_{\text{off}} > \tau$  to ensure all time-varying information is captured during offline precomputation.

The usefulness of periodic systems is that methods and guarantees applicable over time intervals within the offline computation horizon  $T_{\text{off}}$ , are also applicable to intervals outside of it, where the dynamics are equivalent since the system is periodic. Fundamentally, the reason for this is because the value function  $V(r, t)$  computed explicitly over  $t \in [k\tau, k\tau + T_{\text{off}}]$  for  $k \in \mathbb{N}_0$  is equivalent to  $V(r, t - k\tau)$  for  $t \in [k\tau, k\tau + T_{\text{off}}]$ , where  $t - k\tau \in [0, T_{\text{off}}]$  holds. Thus, by computing  $V(r, t)$  over  $[0, T_{\text{off}}]$ , we have essentially obtained the value function (and optimal controller) computed over any interval  $t \in [k\tau, k\tau + T_{\text{off}}]$  for  $k \in \mathbb{N}_0$  for free, without explicitly performing the computation. This idea underlies the analysis in Sec. VI-C.

To relate arbitrary times to the offline computation interval, we introduce the *earliest equivalent interval map*:

$$\mathcal{M}_\tau([t_a, t_b]) := [t_a - \lfloor t_a/\tau \rfloor \tau, t_b - \lfloor t_a/\tau \rfloor \tau]$$

for  $t_b \geq t_a \geq 0$ , where  $\lfloor \cdot \rfloor$  is the floor function. It maps an arbitrary time interval to the earliest interval after zero where the system dynamics are equivalent. Note that when  $t_b - t_a \leq T_{\text{off}} - \tau$ , the earliest equivalent map is within the offline computation interval, i.e.  $\mathcal{M}_\tau([t_a, t_b]) \subseteq [0, T_{\text{off}}]$ . Moreover, let  $\mathcal{M}_\tau^i(t_a, t_b) = \min(\mathcal{M}_\tau([t_a, t_b]))$  and  $\mathcal{M}_\tau^f(t_a, t_b) = \max(\mathcal{M}_\tau([t_a, t_b]))$  denote the initial and final times in the earliest equivalent interval respectively. These functions will be used throughout Sec. VI-B and VI-C.

Just like Sec. V, we assume the constraints  $\mathcal{C}(t)$  for  $t \in [0, T_{\text{run}}]$  are updated online at time instances  $\{t_i^{\text{updated}}\}_{i=0}^{N_{\text{update}}}$ , satisfying (21).

### B. Method for Replanning and Tracking for Periodic Systems

Our method for real-time trajectory replanning and tracking for periodic systems subject to disturbances  $d \in \mathcal{D}$  over a bounded time interval  $t \in [0, T_{\text{run}}]$  is summarised in Alg. 4. The logic in Alg. 4 is similar to Alg. 3 in the sense that they both involve repeated planning and tracking, as illustrated in Fig. 2, but the concrete steps are quite different. Rather than describing every line, we instead bring attention to these differences.

Firstly, we allow for  $T_{\text{run}} \geq 0$ , enabling task horizons longer than the offline computation interval  $T_{\text{off}}$ . Moreover, there is a new input  $T' < T_{\text{off}} - \tau$ , which is the *planning horizon* — that is, the time horizon over which trajectory planning occurs, which ensures that the interval over which trajectories are planned can be successfully mapped back to the offline computation horizon. Subsequently, in line 10, we force replanning when  $t - t_k \geq T'$ . In line 13,  $(t_i, t_f)$  are computed. They correspond to the initial and final time after mapping the interval  $[t_k, t_k + T']$  into the offline computation horizon. The initial time  $t_i$  is subsequently used to specify the minimum value level  $\underline{V}(s_k, t_i)$  in line 14, and the initial planning system sublevel set  $\mathcal{T}_p(s_k, t_i; c_k)$  in line 15. *PlanTrajectoryOverInterval* is applied to generate a trajectory  $\hat{\xi}_{h,k}^{\mathcal{M}}(t')$  over the remapped interval  $t' \in [t_i, t_f]$  in



**Algorithm 4** Safe Trajectory Replanning and Tracking for Periodic Systems

---

```

1: Input:  $L \in \mathbb{R}^{n_r \times n_s}$ ,  $M \in \mathbb{R}^{n_r \times n_p}$ ,  $T_{\text{off}} > \tau$ ,  $f(t, s, u_s, d)$  for  $t \in [0, T_{\text{off}}]$ ,  $h(p, d)$ ,  $\mathcal{G} \subseteq \mathcal{S}$ ,  $T_{\text{run}} \geq 0$ ,  $T' \leq T_{\text{off}} - \tau$ 
2: Compute  $V(r, t)$ ,  $t \in [0, T_{\text{off}}]$  and  $u_s^*(r, t)$ ,  $t \in [0, T_{\text{off}}]$  using  $\text{OfflineStage}(L, M, C, f, h)$  from Alg. 1
3:  $k \leftarrow -1$ ,  $t_{-1} \leftarrow 0$ 
4: for  $t \in [0, T_{\text{run}}]$  do
5:   Measure current tracking system state  $s$  from (1)
6:   if  $s \in \mathcal{G}$  then
7:     Return (goal reached)
8:   end if
9:   Update constraints  $\mathcal{C}_{\text{curr}} \leftarrow \mathcal{C}(t)$ 
10:  if ( $\mathcal{C}_{\text{curr}}$  has changed) OR (choose to replan at time  $t$ ) OR ( $t - t_k \geq T'$ ) then
11:     $k \leftarrow k + 1$ 
12:     $(t_k, s_k) \leftarrow (t, s)$ 
13:     $(t_i, t_f) \leftarrow (\mathcal{M}_\tau^i(t_k, t_k + T'), \mathcal{M}_\tau^f(t_k, t_k + T'))$ 
14:    Choose  $c_k \geq \underline{V}(s_k, t_i)$  from (14)
15:    Choose  $p_k \in \mathcal{T}_p(s_k, t_i; c_k)$  from (13)
16:     $\hat{\xi}_{h,k}^M(t')$ ,  $t' \in [t_i, t_f] \leftarrow \text{PlanTrajectoryOverInterval}(V, h, t_i, s_k, t_f, \mathcal{C}_{\text{curr}}, \mathcal{G}, c)$  from Alg. 2
17:     $\hat{\xi}_{h,k}(t') \leftarrow \hat{\xi}_{h,k}^M(\mathcal{M}_\tau^f(t_k, t'))$ ,  $t' \in [t_k, t_k + T']$ 
18:  end if
19:   $t_c \leftarrow \mathcal{M}_\tau^f([t_k, t])$ 
20:   $\hat{\xi}_h^R(t) \leftarrow \hat{\xi}_{h,k}(t)$ 
21:  Compute  $u_s \leftarrow u_s^*(Ls - M\hat{\xi}_h^R(t), t_c)$  following (7)
22:  Apply  $u_s$  as control input to tracking system (1)
23: end for

```

---

line 16. The planned trajectory is then mapped back to the original interval, yielding  $\hat{\xi}_{h,k}(t')$ ,  $t' \in [t_k, t_k + T']$  in line 17.

The final differences are lines 19, 20, and 21. The current time  $t$  is mapped back to the offline interval to obtain  $t_c$  in line 19, the planned trajectory in the offline interval  $\hat{\xi}_{h,k}^M(t_c)$  is used to obtain the current state of the replanned trajectory  $\hat{\xi}_h^R(t)$  in line 20, and these are subsequently used to compute the control in line 21.

All of the changes described between Alg. 3 and Alg. 4 address the need for objects related to the offline computation — i.e. the value function  $V$  (5) or optimal tracking controller  $u_s^*$  (7) — to have their times mapped back to the offline interval where the system dynamics are equivalent. Making this change allows us to overcome the limitations in Sec. V and deal with the case where  $T_{\text{run}} > T_{\text{off}}$  for periodic systems. Aside from this, Alg. 4 inherits the same properties as Alg. 3, including the requirement for careful design of the replanning criteria in line 10, flexibility to choose  $c_k$  and  $p_k$  in lines 14 and 15, and the capability of switching between the optimal tracking controller and an arbitrary controller in line 21.

### C. Theoretical Guarantees for Replanning and Tracking in Periodic Systems

In this section, we theoretically analyze the usage of Alg. 4 for controlling the system (1) with periodic dynamics (28) by providing Cor. 5. Our analysis progresses similarly to Sec. V-C. Before describing this result, we introduce the required mathematical objects.

Let  $K \in \mathbb{N}$  denote the number of times trajectory replanning occurs. Consider a sequence of planning system states  $\{p_k\}_{k=0}^{K-1} \subseteq \mathcal{P}$ , and a sequence of increasing time instances  $\{t_k\}_{k=0}^K$  where replanning occurs, satisfying  $t_0 = 0$ ,  $t_K = T_{\text{run}}$ . Assume  $\{t_i^{\text{update}}\}_{i=0}^{N_{\text{update}}} \subseteq \{t_k\}_{k=0}^K$ , such that replanning occurs whenever constraints are updated. We define the *replanned trajectory*  $\xi_h^R: [0, T_{\text{run}}] \rightarrow \mathcal{P}$  as the concatenation of trajectories defined on the sub-intervals  $[t_k, t_{k+1})$  for  $0 \leq k \leq K-1$ , corresponding to the worst-case planned trajectory (17) over the interval  $[\mathcal{M}_\tau^i(t_k, t_{k+1}), \mathcal{M}_\tau^f(t_k, t_{k+1})]$  with initial planning system state  $p_k$  and tracking system state  $s_k$ :

$$\xi_h^R(t) := \xi_h^*(\mathcal{M}_\tau^f(t_k, t); p_k, s_k, \mathcal{M}_\tau^i(t_k, t)), \quad (29)$$

$$t \in [t_k, t_{k+1}), \quad 0 \leq k \leq K-1.$$

Here,  $s_k$  is recursively defined as

$$s_k := \xi_f^*(\mathcal{M}_\tau^f(t_{k-1}, t_k); s_{k-1}, p_{k-1}, \mathcal{M}_\tau^i(t_{k-1}, t_k)).$$

and  $s_0$  is the state of the tracking system at time  $t=0$ .

Next, define the *replanned tracking system controls*  $u_s^R: [0, T_{\text{run}}] \rightarrow \mathcal{U}_s$  as the concatenation of the optimal tracking system controls (12), computed over the remapped intervals in the offline computation horizon that correspond to each sub-interval between replanning time instances:

$$u_s^R(t) := u_s^*(\mathcal{M}_\tau^f(t_k, t); Ls_k - Mp_k, \mathcal{M}_\tau^i(t_k, t)),$$

$$t \in [t_k, t_{k+1}), \quad 0 \leq k \leq K-1.$$

Moreover, we define the *replanned disturbances* as the concatenation of the worst-case disturbances (11) computed over the remapped intervals in the offline computation horizon that correspond to each sub-interval between replanning time instances:

$$d^R(t) := d^*(\mathcal{M}_\tau^f(t_k, t); Ls_k - Mp_k, \mathcal{M}_\tau^i(t_k, t)),$$

$$t \in [t_k, t_{k+1}), \quad 0 \leq k \leq K-1.$$

Finally, the *replanned tracking system trajectory* is defined as the trajectory of the tracking system starting from state  $s_0$  at time 0 under the replanned tracking system controls (23) and disturbances (24):

$$\xi_f^R(t) := \xi_f(t; s_0, 0, u_s^R(\cdot), d^R(\cdot)), \quad t \in [0, T_{\text{run}}].$$

For all time  $t \in [0, T_{\text{run}}]$ ,  $\xi_h^R(t)$ ,  $u_s^R(t)$ , and  $d^R(t)$ , are uniquely determined by  $s_0$ ,  $\{p_k\}_{k=0}^{K-1}$  and  $\{t_k\}_{k=0}^K$ , where  $k'(t) = \max\{k \in \mathbb{N}_0 : t_k \leq t\}$ .

With these mathematical objects defined, we are now ready to provide Cor. 5, which says that, for each replanning step  $k \in \{0, \dots, K-1\}$  and associated time interval  $[t_k, t_{k+1})$ , as long as this time interval can be mapped back to the offline computation interval (30), the corresponding value level  $c_k$  is sufficiently large (31) and the initial planning system state  $p_k$  in this interval is inside the planning system sublevel set  $\mathcal{T}_p(\xi_f^R(t_k), \mathcal{M}_\tau^i(t_k, t_{k+1}); c_k)$  (32), then at any time in this interval that the replanned trajectory  $\xi_h^R(t)$  is within the planning system constraints  $\mathcal{F}_{\mathcal{G}}(\mathcal{M}_\tau^f(t_k, t); c_k)$  (goal  $\mathcal{F}_{\mathcal{G}}(\mathcal{M}_\tau^f(t_k, t); c_k)$ ), the replanned tracking system trajectory  $\xi_f^R(t)$  satisfies the constraints  $\mathcal{C}(t)$  (33) (goal  $\mathcal{G}$  (34)).

**Corollary 5.** Let  $\{c_k\}_{k=0}^{K-1} \subseteq \mathbb{R}$  be a sequence of value levels. Suppose for all  $k \in \{0, \dots, K-1\}$ , the following hold:

$$t_{k+1} - t_k \leq T_{\text{off}} - \tau, \quad (30)$$

$$c_k \geq \underline{V}(\xi_f^R(t_k), \mathcal{M}_\tau^i(t_k, t_{k+1})), \quad (31)$$

$$p_k \in \mathcal{T}_p(\xi_f^R(t_k), \mathcal{M}_\tau^i(t_k, t_{k+1}); c_k). \quad (32)$$

Then, for any  $k \in \{0, \dots, K-1\}$  and any  $t \in [t_k, t_{k+1})$ , we have:

$$\xi_h^R(t) \in \mathcal{F}_{C(t_k)}(\mathcal{M}_r^f(t_k, t); c_k) \implies \xi_f^R(t) \in \mathcal{C}(t), \quad (33)$$

$$\xi_h^R(t) \in \mathcal{G}_p(\mathcal{M}_r^f(t_k, t); c_k) \implies \xi_f^R(t) \in \mathcal{G}. \quad (34)$$

**Remark 11.** Similar to Cor. 4, Cor. 5 provides sufficient conditions for collision avoidance and/or goal-reaching over the task horizon supposing that within each sub-interval  $[t_k, t_{k+1})$  for  $k \in \{0, \dots, K-1\}$ , optimal tracking system controls  $u_s^*$  (12) and worst-case disturbances  $d^*$  (11) are applied to (1), and the worst-case planning system controls  $u_p^*$  (10) are applied to (2), after accounting for the reinitialization of the planning system state at  $p_k$ . However, these optimal controls and disturbances are solved over remapped intervals inside the offline computation horizon, as opposed to the true sub-interval. Since the dynamics of the tracking system are equivalent across both due to the periodic nature of the system, this extra complexity allows us to provide guarantees for  $T_{\text{run}} > T_{\text{off}}$ , without requiring the solution of the HJ differential game beyond  $T_{\text{off}}$ . The proof of Cor. 5 naturally follows by applying Thm. 3 for each remapped interval corresponding to each sub-interval of the task horizon. Thus, for the same reasons pointed out in Remark 5, the results in Cor. 5 also hold when the optimal tracking controller (7) for each remapped interval, and arbitrary planning system controls and disturbances taking values in  $\mathcal{U}_p$  and  $\mathcal{D}$  are applied in each sub-interval of the task horizon.

The connections between Alg. 4 and the guarantees in Cor. 5 follow similarly to that of Alg. 3 and Cor. 4. In particular, the planning system controls for the replanned trajectories take values in  $\mathcal{U}_p$ , the tracking system controls are generated by the optimal tracking controller (line 21), and the disturbances take values in  $\mathcal{D}$ ; therefore, referring to Remark 11, the guarantees in Cor. 5 apply to the system controlled by Alg. 4 as long as the conditions in Cor. 5 hold after replacing  $\xi_h^R(t)$  and  $\xi_f^R(t_k)$  with  $\hat{\xi}_h^R(t)$  and  $s_k$  from Alg. 4. This can be verified by comparing line 14 in Alg. 3 to (31), line 15 to (32), and  $\hat{\xi}_f^R(t) = \hat{\xi}_{h,k}^R(t)$  (which satisfies C3 and C4 via line 16) to (33) and (34). Following the same logic as Sec. V-C, (33) and (34) can therefore be applied to find that constraint satisfaction and (optionally) goal-reaching are achieved between replanning time instances, implying Alg. 4 achieves these guarantees over the entire task horizon.

## VII. EXAMPLE: AUV SUBJECT TO WAVE DISTURBANCES

This section explains how the methods in this paper can be implemented on a numerical example of an AUV, and showcases its performance. Different ways of modeling an AUV subject to wave disturbances are provided in Sec. VII-A, and in Sec. VII-B we describe how to interface these models with the OfflineStage (Alg. 1) component of our framework. In Sec. VII-C, a concrete implementation of the planning procedure in Alg. 2 is described, and its benefits are highlighted. Sec. VII-D showcases how the combined replanning and tracking procedure in Alg. 3 can successfully be used to guide the AUV through an *a priori* unknown environment, and how flexible replanning leads to performance improvements. Since Alg. 3 requires  $T_{\text{run}} \leq T_{\text{off}}$ , in Sec. VII-D we show that Alg. 4 can be applied to the AUV when  $T_{\text{run}} \gg T_{\text{off}}$  by exploiting periodicity.

### A. Modeling the AUV Subject to Wave Disturbances

In this section, we first provide a time-varying, state-dependent system model of an AUV in plane progressive waves derived based

on [28]. This model is lifted from [18, eq. 9], which we refer to as the tracking system model for Case 1. Following this, we derive two approximations of this tracking system model which differ based on the fidelity of the wave disturbance model. In particular, Case 2 corresponds to a time-varying, state-independent wave approximation, and Case 3 is a time- and state-independent approximation. These cases will be used to demonstrate the benefits and limitations of including or discarding model information in our methods.

**1) Case 1: Tracking System for Time-Varying, State-Dependent Wave Model:** Consider the following model  $\dot{s} = \mathbf{f}(s, u_s, d^{\text{nom}}, d^{\text{wave}})$  of an AUV subject to both a nominal and wave disturbance:

$$\begin{aligned} \dot{x} &= u_r + d_x^{\text{wave}} + d_x \\ \dot{z} &= w_r + d_z^{\text{wave}} + d_z \\ \dot{u}_r &= (m - X_{\dot{u}})^{-1}((\bar{m} - m)d_u^{\text{wave}} \\ &\quad - (X_u + X_{|u|u}|u_r|)u_r + T_A) + d_u \\ \dot{w}_r &= (m - Z_{\dot{w}})^{-1}((\bar{m} - m)d_z^{\text{wave}} - (-g(m - \bar{m})) \\ &\quad - (Z_w + Z_{|w|w}|w_r|)w_r + T_B) + d_w. \end{aligned} \quad (35)$$

Here,  $s = (x, z, u_r, w_r)$  is the state, where  $(x, z)$  is the AUV position ( $z$  pointing downwards) and  $(u_r, w_r)$  is the relative velocity between the AUV and water flow. The control input is  $u_s = (T_A, T_B)$ ,  $d^{\text{wave}} = (d_x^{\text{wave}}, d_z^{\text{wave}}, d_u^{\text{wave}}, d_w^{\text{wave}})$  is the wave disturbances, and  $d^{\text{nom}} = (d_x, d_z, d_u, d_w)$  is other unmodeled, bounded disturbances. Model parameters include the AUV mass  $m$ , the displaced fluid mass  $\bar{m}$ , the additive masses  $X_{\dot{u}}, Z_{\dot{w}}$ , the linear damping factors  $X_u, Z_u$ , the quadratic damping factors  $X_{|u|u}, Z_{|w|w}$ , and gravity  $g$ .

Let  $\mathbf{d}^{\text{tvsd}}(t, s) = (W_x(x, z, t), W_z(x, z, t), A_x(x, z, t), A_z(x, z, t))$  represent the plane-progressive wave model, where  $(W_x, W_z)$  and  $(A_x, A_z)$  are the velocity and acceleration models, given by

$$\begin{aligned} \begin{bmatrix} W_x(x, z, t) \\ W_z(x, z, t) \end{bmatrix} &= \tilde{A}\omega e^{-kz} \begin{bmatrix} \cos(kx - \omega t) \\ -\sin(kx - \omega t) \end{bmatrix}, \\ \begin{bmatrix} A_x(x, z, t) \\ A_z(x, z, t) \end{bmatrix} &= \tilde{A}\omega^2 e^{-kz} \begin{bmatrix} \sin(kx - \omega t) \\ \cos(kx - \omega t) \end{bmatrix}. \end{aligned} \quad (36)$$

Here,  $\tilde{A}$  is the wave amplitude,  $\omega$  is the wave frequency, and  $k$  is the wavenumber. Note that  $\mathbf{d}^{\text{tvsd}}$  is both time-varying and state-dependent. The Case 1 tracking system model is then obtained by choosing the wave disturbance  $d^{\text{wave}}$  in the AUV model (35) as  $\mathbf{d}^{\text{tvsd}}(s, t)$ , producing a time-varying system model  $\mathbf{f}(t, s, u_s, d) = \mathbf{f}(s, u_s, d^{\text{nom}}, \mathbf{d}^{\text{tvsd}}(t, s))$  for  $t \in [0, T]$ , with  $d = d^{\text{nom}}$ .

**2) Case 2: Tracking System for Time-Varying, State-Independent Wave Model:** In this case, we consider a time-varying, state-independent approximation of the plane-progressive wave model  $\mathbf{d}^{\text{tvsi}}(t, s)$  with the following structure:

$$\mathbf{d}^{\text{tvsi}}(t, d^{\text{tvsi}}) = \begin{bmatrix} \tilde{A}_W^{\text{tvsi}} \cos(\phi_W - \omega t) + d_{W,x}^{\text{tvsi}} \\ -\tilde{A}_W^{\text{tvsi}} \sin(\phi_W - \omega t) + d_{W,z}^{\text{tvsi}} \\ \tilde{A}_A^{\text{tvsi}} \sin(\phi_A - \omega t) + d_{A,x}^{\text{tvsi}} \\ \tilde{A}_A^{\text{tvsi}} \cos(\phi_A - \omega t) + d_{A,z}^{\text{tvsi}} \end{bmatrix}, \quad (37)$$

where  $d^{\text{tvsi}} = (d_{W,x}^{\text{tvsi}}, d_{W,z}^{\text{tvsi}}, d_{A,x}^{\text{tvsi}}, d_{A,z}^{\text{tvsi}})$  are bounded uncertainties satisfying  $|d_{W,x}^{\text{tvsi}}|, |d_{W,z}^{\text{tvsi}}| \leq D_W^{\text{tvsi}}$  and  $|d_{A,x}^{\text{tvsi}}|, |d_{A,z}^{\text{tvsi}}| \leq D_A^{\text{tvsi}}$ . Here,  $\tilde{A}_W^{\text{tvsi}}, \tilde{A}_A^{\text{tvsi}}, \phi_W, \phi_A, D_W^{\text{tvsi}}, D_A^{\text{tvsi}}$  are parameters that need to be appropriately selected so that  $\mathbf{d}^{\text{tvsd}}(t, s)$  is conservatively approximated by  $\mathbf{d}^{\text{tvsi}}(t, d^{\text{tvsi}})$  over some bounded region of interest

$\Xi \subseteq \mathbb{R}^2$  in the  $x$ - $z$  space, and time interval  $[0, T]$ . They are chosen so that for all  $(x, z) \in \Xi$  and  $t \in [0, T]$ ,

$$\begin{bmatrix} W_x(x, z, t) \\ W_z(x, z, t) \\ A_x(x, z, t) \\ A_z(x, z, t) \end{bmatrix} \in \left\{ \begin{bmatrix} \tilde{A}_W^{\text{tvis}} \cos(\phi_W - \omega t) \\ \tilde{A}_W^{\text{tvis}} (-\sin(\phi_W - \omega t)) \\ \tilde{A}_A^{\text{tvis}} \sin(\phi_A - \omega t) \\ \tilde{A}_A^{\text{tvis}} \cos(\phi_A - \omega t) \end{bmatrix} \right\} \oplus [-D_W^{\text{tvis}}, D_W^{\text{tvis}}]^2 \times [-D_A^{\text{tvis}}, D_A^{\text{tvis}}]^2 \quad (38)$$

holds with the smallest  $\tilde{A}_W^{\text{tvis}}, \tilde{A}_A^{\text{tvis}}, D_W^{\text{tvis}}, D_A^{\text{tvis}} > 0$ .

The Case 2 tracking system model is then formulated as the time-varying system  $f(t, s, u_s, d) = \underline{f}(s, u_s, d^{\text{nom}}, \underline{d}^{\text{tvis}}(t, d^{\text{tvis}}))$  for  $t \in [0, T]$ , with  $d = (d^{\text{nom}}, d^{\text{tvis}})$ .

**3) Case 3: Tracking System for Time- and State-Independent Wave Model:** For Case 3, the wave model is simply an additive, bounded disturbance  $d^{\text{tisi}} = (d_{W,x}^{\text{tisi}}, d_{W,z}^{\text{tisi}}, d_{A,x}^{\text{tisi}}, d_{A,z}^{\text{tisi}})$ , satisfying  $|d_{W,x}^{\text{tisi}}|, |d_{W,z}^{\text{tisi}}| \leq D_W^{\text{tisi}}$  and  $|d_{A,x}^{\text{tisi}}|, |d_{A,z}^{\text{tisi}}| \leq D_A^{\text{tisi}}$ . The parameters  $D_W^{\text{tisi}}, D_A^{\text{tisi}}$  are chosen as the smallest values capturing uncertainty associated with moving from  $\underline{d}^{\text{tvsd}}$  to a time-invariant approximation, i.e.  $\underline{d}^{\text{tvsd}}(t, (x, z, u_r, w_r)) \in [-D_W^{\text{tisi}}, D_W^{\text{tisi}}]^2 \times [-D_A^{\text{tisi}}, D_A^{\text{tisi}}]^2$  must be satisfied for all  $(x, z, u_r, w_r) \in \Xi \times \mathbb{R}^2$  and  $t \in [0, T]$ . The Case 3 tracking system model is then formulated as a time-invariant system  $f(s, u_s, d) = \underline{f}(s, u_s, d^{\text{nom}}, d^{\text{tisi}})$  over  $t \in [0, T]$ , with  $d = (d^{\text{nom}}, d^{\text{tisi}})$ .

## B. Interfacing OfflineStage with the AUV Model

In this section, we describe three different ways of selecting the inputs  $L, M, C, f, h$  for OfflineStage based on Cases 1-3 in VII-A. In all cases, we choose the planning system  $h$  as the single integrator  $\dot{p} = h(p, u_p) = (\dot{x}_p, \dot{z}_p) = (u_{p,x}, u_{p,z})$ , where  $p = (x_p, z_p)$  is the planning system state and  $u_p = (u_{p,x}, u_{p,z})$  is the planning system control.

**1) Case 1: Full Wave:** Recall the Case 1 tracking system model  $f(t, s, u_s, d) = \underline{f}(s, u_s, d^{\text{nom}}, \underline{d}^{\text{tvsd}}(t, s))$ , with  $d = d^{\text{nom}}$ . Here, we choose  $L = \begin{bmatrix} I_2 & 0_{2 \times 2} & I_2 \\ 0_{2 \times 2} & I_2 & 0_{2 \times 2} \end{bmatrix}^\top$  and  $M = [I_2 \quad 0_{2 \times 4}]^\top$ , which following (3) formulates a 6D relative system with states  $r = (x_\alpha, z_\alpha, u_r, w_r, x, z)$ , where  $(x_\alpha, z_\alpha) = (x - x_p, z - z_p)$  is the tracking error between the tracking and planning positions. The position  $(x, z)$  is needed in  $r$  since  $\underline{d}^{\text{tvsd}}(t, s)$  depends on it, in contrast to Cases 2 and 3. This produces different value functions, requiring different on-line planning and tracking procedures to handle this change. The error matrix  $C$  is then chosen as  $C = [I_2 \quad 0_{2 \times 4}]$ . Moreover, line 3 in Alg. 1 was carried out using OptimizedDP [23] due to its 6D support.

**2) Case 2 and 3: Time-Varying/Time-Invariant Wave Approximation:** Recall the Case 2 tracking system model  $f(t, s, u_s, d) = \underline{f}(s, u_s, d^{\text{nom}}, \underline{d}^{\text{tvis}}(t, d^{\text{tvis}}))$ , with  $d = (d^{\text{nom}}, d^{\text{tvis}})$ , and the Case 3 model  $f(s, u_s, d) = \underline{f}(s, u_s, d^{\text{nom}}, d^{\text{tisi}})$ , with  $d = (d^{\text{nom}}, d^{\text{tisi}})$ . For both cases, we choose  $L = I_4$  and  $M = [I_2 \quad 0_{2 \times 2}]^\top$ , which following (3) formulates a 4D relative system with states  $r = (x_\alpha, z_\alpha, u_r, w_r)$ . Unlike Case 1,  $(x, z)$  is not needed in  $r$  since both  $\underline{d}^{\text{tvis}}(t, d^{\text{tvis}})$  and  $d^{\text{tisi}}$  are state-independent.  $C$  is then chosen as  $C = [I_2 \quad 0_{2 \times 2}]$ . Note that the offline computation of Case 3 can be viewed as a naive extension of the offline procedure in FaSTrack to time-varying systems by uniformly bounding the waves.

**Remark 12.** Although we pointed out in Sec. III-B that the consideration of worst-case planning system behavior is usually conservative for most problems, we find it is reasonable in this example. Here, it corresponds to the tracking system  $f$  trying to catch up to a planning

system  $h$  that opposes the direction of the waves in (36), which can occur frequently due to the periodic nature of these waves.

## C. Trajectory Planning for the AUV

Although Alg. 2 abstractly describes the planning procedure, how to implement the procedure remains unclear since we need to compute objects that are defined in continuous time and space. In this section, we describe example implementations of Alg. 2 for the problem of navigating an AUV subject to plane-progressive wave disturbances from an initial state  $s_0 \in \mathcal{S}$  to a goal region  $\mathcal{G}$  while satisfying constraints  $\mathcal{C}$ . These implementations are based on the Case 1-3 AUV models from VII-A and were originally contained in our conference paper [18]. Note that the objects computed in these implementations are discrete time and space approximations of the continuous objects in Alg. 2, and thus not directly compatible with the safety guarantees of this work. Nevertheless, they are practically useful, and we leave considering the effect of approximation errors on guarantees for future work. We assume the AUV is a point mass for simplicity, and that there are only static obstacles  $\mathbb{O} \subseteq \mathbb{R}^2$  in the  $x$ - $z$  plane, such that we have a static constraint  $\mathcal{C} = \mathbb{O}^c \times \mathbb{R}^2$ . However, considering the AUV volume is important for collision avoidance in reality, and it can easily be accounted for using the constraint  $\mathcal{C} = (\mathbb{O} \oplus (-\mathbb{V}))^c \times \mathbb{R}^2$ , where  $\mathbb{V}$  is the AUV volume centered at the tracking system position. We also assume  $\Xi^c \subseteq \mathbb{O}$  since the wave models are only accurate for  $(x, z) \in \Xi$ , and that the goal is determined by position, i.e.  $\mathcal{G} = \mathbb{G} \times \mathbb{R}^2 \subseteq \mathcal{S}$ , where  $\mathbb{G} \subseteq \mathbb{R}^2$  is the goal region in the  $x$ - $z$  plane. In VII-C.1, we describe how to approximately implement the algorithm for the models based on Cases 1-3. In VII-C.2, we briefly comment on the differences.

**1) Implementing Planning Algorithms:** We first describe the implementation of Alg. 2 for Case 1, followed by 2 and 3 simultaneously. Throughout all examples, suppose  $T_{\text{off}} \in [0, T]$ ,  $t_i \in [0, T_{\text{off}}]$ , and  $t_f \in [t_i, T_{\text{off}}]$ . Moreover, suppose  $\mathcal{C}$  and  $\mathcal{G}$  have been provided,  $h$  is the single integrator system from VII-B, and both  $c \geq \underline{V}(s, t)$  and  $p \in \mathcal{T}_p(s, t_i; c)$  are given.

**a) Case 1: Full Wave Model:** Suppose Alg. 1 has been run for Case 1 based on VII-B.1. Let  $\oplus$  denote the Minkowski sum. To implement line 2 of Alg. 2, the planning system constraints can be expressed as  $\mathcal{C}_p(t) = \mathcal{F}_C(t; c) = \mathcal{O}_p^c(t)$ , where  $\mathcal{O}_p$  denotes the *planning system obstacles* and is defined using the set avoidance map from Sec. IV-A as

$$\begin{aligned} \mathcal{O}_p(t) = \mathcal{A}_{C^c}(t; c) &= \bigcup_{(x, y) \in \mathbb{O}(u_r, w_r) \in \mathbb{R}^2} \mathcal{T}_p(s, t; c) \\ &= \bigcup_{(x, z) \in \mathbb{O}} \{(x, z)\} \oplus (-\mathcal{B}_e^{\text{full}}(x, z, t; c)) \end{aligned}$$

for  $t \in [t_i, t_f]$ , where  $\mathcal{B}_e^{\text{full}}(x, z, t; c) := \{(x_\alpha, z_\alpha) : V((x_\alpha, z_\alpha, u_r, w_r, x, z), t) \leq c \text{ for some } u_r, w_r\}$  denotes the tracking error bound (TEB) for the full wave model. We can approximately compute  $\mathcal{O}_p(t)$  by setting up a grid of discrete cells representing  $x, z$  points to iterate over, and following Alg. 5. In Alg. 5,  $k$  refers to the time step,  $N$  is the horizon length,  $t_k = t_i + kT_s$  is the discretized time, and  $T_s = (t_f - t_i)/N$  is the time discretization period. Note that  $\mathcal{C}_p(t_k)$  could be obtained by taking the complement of the output  $\mathcal{O}_p(t_k)$ . We will keep  $\mathcal{O}_p(t_k)$  since it is a more convenient representation in this example.



**Algorithm 5** Computing Planning System Obstacles for Case 1 AUV Example

---

```

1: Input:  $\mathbb{O}$ ,  $N$ ,  $\mathcal{B}_e^{\text{full}}(x, z, t_k; c)$  for  $k \in \{0, \dots, N\}$ ,  $t_i$ ,  $T_s$ 
2: for  $k \in \{0, \dots, N\}$  do
3:    $t_k = t_i + kT_s$ 
4:    $\mathcal{O}_p(t_k) \leftarrow \emptyset$ 
5:   for  $(x, z) \in \mathbb{O}$  do
6:     for  $(x_\alpha, z_\alpha) \in \mathcal{B}_e^{\text{full}}(x, z, t_k; c)$  do
7:        $\mathcal{O}_p(t_k) \leftarrow \mathcal{O}_p(t_k) \cup \{(x, z) - (x_\alpha, z_\alpha)\}$ 
8:     end for
9:   end for
10: end for
11: Output:  $\mathcal{O}_p(t_k)$  for  $k \in \{0, \dots, N\}$ 

```

---

Similarly,  $\mathcal{G}_p(t)$ ,  $t \in [t_i, t_f]$  can be formulated as  $\mathcal{G}_p(t) = (\bigcup_{(x,z) \in \mathbb{G}^c} \{(x, z)\} \oplus (-\mathcal{B}_e^{\text{full}}(x, z, t; c)))^c$ , so line 3 of Alg. 2 is approximately implemented by inputting  $\mathcal{G}$  instead of  $\mathcal{O}$  into Alg. 5, and taking the complement of the output.

Finally, we implement line 4 of Alg. 2 by solving a discrete-time trajectory optimization problem in (39), where  $h_d$  denotes the Euler discretization of  $h$  with period  $T_s$ . Conditions C1-C4 are represented in the constraints of this problem.

$$\begin{aligned}
 & \min_{\{p_k\}_{k=0}^N, \{u_{p,k}\}_{k=0}^{N-1}} \sum_{k=0}^{N-1} ((p_k - p_{\text{ref}})^\top Q (p_k - p_{\text{ref}}) \\
 & \quad + u_{p,k}^\top R u_{p,k}) + (p_N - p_{\text{ref}})^\top Q (p_N - p_{\text{ref}}) \\
 \text{s.t. } & p_{k+1} = h_d(p_k, u_{p,k}), u_{p,k} \in \mathcal{U}_p, p_k \notin \mathcal{O}_p(t_k), \\
 & p_0 = p, p_N \in \mathcal{G}_p(t_N).
 \end{aligned} \tag{39}$$

Here,  $\{p_k\}_{k=0}^N$  and  $\{u_{p,k}\}_{k=0}^{N-1}$  are planning system states and controls over the horizon respectively, and  $p_{\text{ref}} \in \mathbb{R}^2$  is a reference planning system state chosen to satisfy  $p_{\text{ref}} \in \mathcal{G}_p(t_N)$ . The weight matrices  $Q$  and  $R$  penalize the tracking error and control inputs. The obstacle avoidance constraint  $p_k \notin \mathcal{O}_p(t_k)$  can be practically implemented using optimization-based collision avoidance constraints [29].

Note that here, the computation of the planning system obstacles and the goal is the same as [18, IV.B], differing from FaSTrack [16]. This enables the capturing of spatial information in the wave model.

*b) Case 2 and 3: Time-Varying/Invariant Wave Approximation:*

Suppose Alg. 1 has been run for Case 2 or 3 based on VII-B.2. Unlike in Case 1,  $\mathcal{O}_p(t_k)$  can be simply formulated as  $\mathcal{O}_p(t_k) = \mathbb{O} \oplus (-\mathcal{B}_e^{\text{approx}}(t_k; c))$  for  $k \in \{0, \dots, N\}$ , where  $\mathcal{B}_e^{\text{approx}}(t_k; c) := \{(x_\alpha, z_\alpha) : V((x_\alpha, z_\alpha, u_r, w_r), t) \leq c \text{ for some } u_r, w_r \in \mathbb{R}\}$  is the TEB for the approximate wave model.  $\mathcal{B}_e^{\text{approx}}(t; c)$  can be computed using ToolboxLS [30], and further approximated with the minimum bounding ball for convenience. Its implementation is simpler than Case 1 since  $\mathcal{B}_e^{\text{approx}}(t; c)$  is independent of  $x, z$ . The planning system goal is also easily obtained via  $\mathcal{G}_p(t_k) = \mathbb{G} \ominus \mathcal{B}_e^{\text{approx}}(t_k; c)$ , where  $\ominus$  denotes the Minkowski difference. In these examples, the computations of the planning system obstacles and goals are the same as [18, IV.A], and coincide with FaSTrack [16]. A trajectory that avoids planning system obstacles and enters the goal region is then computed in line 4 by solving the MPC problem in (39) using the new  $\mathcal{O}_p(t_k)$  and  $\mathcal{G}_p(t_k)$ .

**2) Simulating Planning Algorithms:** This section aims to briefly explain the difference in performance between the planned trajectories across Cases 1, 2, and 3. We qualitatively describe these

differences based on the numerical example in our conference paper [18, Sec. V], which we refer the reader to for full details. Note that Cases 1, 2, and 3 in this paper correspond to Cases 3, 2, and 1 (reversed order) in [18]. We also briefly discuss the importance of choosing an appropriate case for real-time implementation.

Our findings were that the planned trajectory for Case 1 reaches the planning system goal region in the shortest time, followed by Case 2, and lastly Case 3. This order is explained by the fact that as we move from Case 1 to Case 2, spatial information for  $\underline{d}^{\text{tsd}}(t, s)$  is moved to the unmodelled uncertainty in  $d^{\text{tsi}}$ , and both time and spatial information is captured in  $d^{\text{tsi}}$  for Case 3. The greater the magnitude of the unmodelled uncertainty, the larger the planning system obstacles and the smaller the goal, producing conservative trajectories and slower goal-reaching. These results highlight the advantages of our time-varying method, compared to naively extending FaSTrack by uniformly bounding the disturbances.

Despite the benefit of using Case 1, there are computational issues to consider. Firstly, OfflineStage in Alg. 1 is more computationally expensive due to the curse of dimensionality in dynamic programming, since it involves a 6D model, versus Cases 2 and 3 which are 4D. For OfflineStage to be tractable in Case 1, a coarser discretization is required, decreasing the accuracy of the approximated value function. Moreover, the generation of planning system obstacles also becomes more complex and expensive (see Alg. 5 versus Sec. VII-C.1.b), which is important because this computation is performed online. Thus, it is critical to carefully consider the choice of model with the view to attain real-time performance. This is further discussed in [18] which focuses on the planning problem.

## D. Replanning and Tracking for the AUV

In this section, we showcase an implementation of Alg. 3 on a simulation of the plane-progressive wave AUV model obtained by combining (35) with (36). The AUV model parameters we consider are obtained from [31], and are  $m = 116$  kg,  $Z_{\dot{w}} = -383$  kg,  $X_{|u|u} = 241.3$  kg/m,  $\bar{m} = 116.2$  kg,  $X_u = 26.9$  kg/s,  $Z_{|w|w} = 265.6$  kg/m,  $X_{\dot{u}} = -167.7$  kg,  $Z_w = 0$  kg/s,  $m = 116$  kg,  $Z_{\dot{w}} = -383$  kg,  $X_{|u|u} = 241.3$  kg/m,  $\bar{m} = 116.2$  kg,  $X_u = 26.9$  kg/s,  $Z_{|w|w} = 265.6$  kg/m,  $X_{\dot{u}} = -167.7$  kg,  $Z_w = 0$  kg/s. Moreover, the disturbances  $d^{\text{nom}} = (d_x, d_z, d_u, d_w)$  are each obtained by applying zero-order hold with time interval 0.2 s to discrete-time i.i.d. random sequences sampled from a uniform distribution over  $[-0.001, 0.001]$ . The plane-progressive wave parameters for (36) we consider are  $\bar{A} = 0.4$  m,  $\omega = 2\pi(0.1)$  rad/s, and  $k = 0.0402$  rad/m.

We aim to control the robot so that starting from  $s_0 = (-1.4, 2.74, 0.0, 0.0)$ , it reaches the goal region  $\mathcal{G}$  whilst avoiding all obstacles in an *a priori* unknown static environment over the region  $\Xi = \{(x, z) \mid x \in [-2, 2], z \in [2, 6]\}$ . We consider  $T_{\text{run}} = 8$  seconds, and  $T_{\text{off}} = 10$  seconds. In this task, we consider an online constraint  $\mathcal{C}(t)$  (described in Sec. V-A) representing known information about the true obstacles at time  $t$ . We assume that the AUV is equipped with a sensor with a range of 1.2 m, and whenever an obstacle partially enters this sensor range at time  $t$ , the entirety of the obstacle has been sensed, so  $\mathcal{C}(t)$  is updated to include this new obstacle information. Additionally,  $\Xi^c$  is considered as a known obstacle for all  $t$ . Moreover, for the HJ game, we assume the tracking/planning system inputs and disturbances are constrained so  $|T_A|, |T_B| \leq 1000$  N,  $|u_{p,x}|, |u_{p,z}| \leq 0.3$  m/s,  $|d_x|, |d_z| \leq 0.001$  m/s, and  $|d_u|, |d_w| \leq$

0.001 m/s<sup>2</sup> are all satisfied. For this example, we consider the Case 2 tracking system model from VII-A.2 for Alg. 3, whose wave model parameters were chosen as  $\tilde{A}_W^{\text{tvs}} = 0.2185$  m/s,  $\tilde{A}_A^{\text{tvs}} = 0.1373$  m/s<sup>2</sup>,  $\phi_W = \phi_A = 0$  rad,  $D_W = 0.03$  m/s, and  $D_A = 0.025$  m/s<sup>2</sup>, since they produce the smallest  $\tilde{A}_W^{\text{tvs}}, \tilde{A}_A^{\text{tvs}}, D_W^{\text{tvs}}, D_A^{\text{tvs}} > 0$  satisfying (38). Hence, the inputs for OfflineStage in line 2, as well as the implementation of PlanTrajectoryOverInterval in line 15, both correspond to Case 2, as described in VII-B.2 in VII-C.1.b. Note that although Case 1 is also compatible, we stick to Case 2 in this section to focus our discussion on the ability of Alg. 3 to handle time-varying systems, rather than the benefit of including spatial information as previously discussed in Sec. VII-C.2.

In Alg. 3, we have the flexibility of choosing when to replan in line 11, and also the value level  $c_k$  and planning system state  $p_k$  according to lines 14 and 15. We repeat the simulation twice under two different implementations of these lines. In Simulation 1, we replan if an unknown obstacle enters the sensor range of the AUV, and fix  $c_k = \underline{V}(s_0, 0) = 0.61$ , and  $p_k = \hat{\xi}_{h,k-1}(t_k)$ . In Simulation 2, in addition to replanning when an unknown obstacle enters the sensor range, we replan every 2 seconds so that the teleportation phenomenon described in Remark 7 occurs more frequently. Additionally, we replan when the AUV enters the region defined by  $z > 3.6$  m. Moreover, we initialize  $p_k$  at the closest point in  $\mathcal{T}_p(s_k, t_k; c_k)$  to  $\mathcal{G}_p(t_k)$ . Initially, we set  $c_k = \underline{V}(s_0, 0) = 0.61$ , but change it to  $c_k = 0.7$  when the AUV enters this region. The reason for this is to increase the size of the region  $\mathcal{T}_p(s_k, t_k; c_k)$  over which  $p_k$  is reinitialized so that the effects of teleportation are further accentuated.

Snapshots of the trajectory for Simulations 1 and 2 and at various time instances where replanning occurs are shown in Figures 4 and 5 respectively. In Simulation 1, replanning due to obstacle detection occurs at  $t = 0, 1.4$  and 5.36 seconds, and the goal is reached at 7.06 seconds. In Simulation 2, replanning due to obstacle detection occurs at  $t = 0, 1.1$  and 3.8 seconds, and the goal is reached at 4.2 seconds. Replanning due to  $z > 3.6$  occurs at  $t = 3.8$  seconds.

In both cases, we observe that whenever an obstacle comes into the sensor range for the tracking system such that  $\mathcal{C}(t)$  changes, a new trajectory is replanned which avoids the corresponding planning system obstacle. This enables the tracking system to successfully avoid all obstacles and reach the goal region, despite the environment being *a priori* unknown. Moreover, the tracking system in Simulation 2 arrives at the goal region earlier than in Simulation 1. This can be attributed to the differences in the implementation of the replanning lines. In particular, due to the selection of a larger  $c_k$  when the AUV satisfies  $z > 3.6$  m and the initialization of  $p_k \in \mathcal{T}_p(s_k, t_k; c_k)$ , the planning system can “teleport” in Simulation 2. This phenomenon can be seen via the discontinuities of the planning system trajectory in Fig. 5. It effectively enables the planning system to traverse through the environment beyond the speed specified when precomputing the  $V(r, t)$  and  $u_s^*(r, t)$ , encouraging the tracking system to catch up and therefore result in faster goal-reaching. This is useful, but note that  $c_k$  can only be increased if there is enough space for the AUV to reach the goal. This is evident at  $t = 3.8$  seconds in Fig. 5, where after increasing  $c_k$ , the planning system obstacles enlarge, leaving no gap for the planning system to pass around the bottom two obstacles if it had not already.

These observations highlight a tradeoff between the degree of “teleportation” and the size of the planning system obstacles that can be exploited for desired performance. It allows our replanning

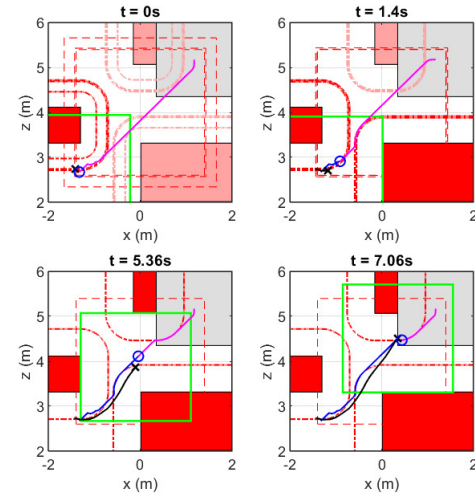


Fig. 4: Snapshots of the tracking and planning systems at time instances where replanning occurs up until the goal is reached, for Simulation 1. Dark red blocks correspond to known obstacles, and the corresponding dark red dotted lines correspond to the planning system obstacles. The dark red dashed squares are due to  $\Xi^c$ , which is included in the known obstacles. Light red blocks correspond to unknown obstacles, and the corresponding light red dotted lines are the unknown future planning system obstacles. The grey box is the goal region. The black ‘x’ is the tracking system position, and the black line is the tracking system trajectory up to the current time. The green box is the sensor range for the tracking system. The blue circle is the planning system, the blue line is the planning system trajectory up to the current time, and the purple line is the current planned trajectory.

mechanism to play a role *similar* to “meta-planning” for FaSTrack [17], which enables safe switching between fast planners for open environments, and slow planners for tight ones. Our method can obtain similar behavior to a fast planner via the selection of a higher  $c_k$ , and a slow planner via the selection of a lower  $c_k$ , with the benefit of only requiring the precomputation of a *single* value function.

To obtain a better understanding of how conservative our method is, we also plotted the value level  $c_k$  versus the evaluation of the value function on the relative state in Fig. 6. This plot illustrates how close the relative system trajectory is to breaching the sublevel set of  $V(r, t)$  corresponding to  $c_k$ , which would cause the violation of safety guarantees. We see that the system comes close to violating the safety guarantees during the times when  $c_k = 0.61$ , suggesting that our method is not too conservative for the considered problem. Although we pointed out in Sec. III-B that the consideration of worst-case planning system behavior is usually conservative for most problems, we find it is reasonable in this example. It can take place when the tracking system  $f$  is catching up to a planning system  $h$  fighting against the waves in (36), which can occur frequently due to the periodic nature of the waves.

**Remark 13.** Fig. 6 suggests our method in Simulation 2 is reasonable, but usually, considering worst-case disturbances/planning is conservative in applications. To get a feel for this, one could gradually increase the size of disturbances and planning system inputs in simulations while plotting the value level  $c_k$  versus the evaluation of

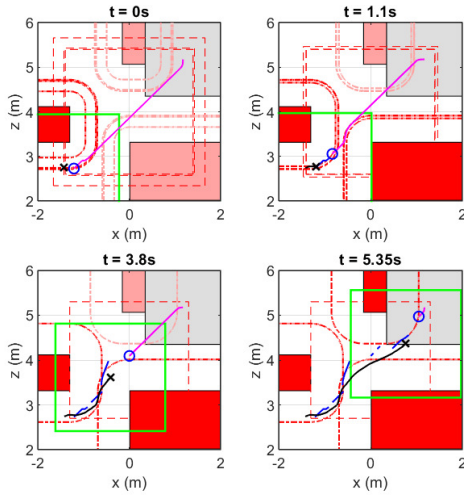


Fig. 5: Snapshots of the tracking and planning system at times where replanning occurs up until the goal is reached for Simulation 2. The meaning of symbols and lines are consistent with Fig. 4.

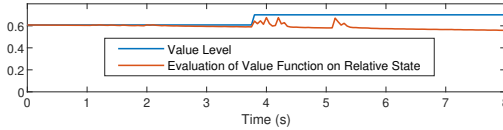


Fig. 6: Comparison plot of value level  $c_k$  versus the evaluation of the value function on the relative state for Simulation 2.

the value function (like in Fig. 6), until the latter breaches the former.

We also tried to simulate replanning and tracking when OfflineStage and PlanTrajectoryOverInterval are implemented according to Case 3, where the tracking system is time-invariant like FaSTrack. Following the modeling procedure in VII-A.2, the Case 3-specific parameters were selected as  $D_W = 0.2319$  and  $D_A = 0.1457$ . However, the task was unsuccessful. Even with the smallest value level  $c_k = \underline{V}(s_0, 0) = 0.67$ , the planning system obstacles were too large, rendering planning infeasible. This is consistent with the findings in Sec. VII-C.2, further highlighting the benefits of incorporating time-varying information inside the tracking system dynamics and using Alg. 3, as opposed to treating it as a bounded disturbance and modeling the tracking system as time-invariant.

The planning procedure in (39) was solved using IPOPT [32] on a laptop with a Core I7-8565U CPU, with a discretization interval of 0.2 s. In Simulation 2, the planning horizon varied from  $N = 104$  at  $t = 0$  to 60 over the simulation based on the length of the planning interval  $[t, T_{\text{run}}]$  at the current time  $t$ . Moreover, the average solve time for Simulation 1 was 1.0 s (averaged over 4 solves, with a standard deviation of 0.4 s), and for Simulation 2 was 1.4 s (averaged over 9 solves, with a standard deviation of 0.9 s). These numbers cannot yet be considered real-time but may be improved with faster hardware. They can also be improved by removing the goal constraint  $p_N \in \mathcal{G}_p(t_N)$  in (39) and shortening the planning interval in line 15 of Alg. 3, as discussed in Remark 3, and additionally applying different tricks for more efficiently solving optimization problems such as warm-starting.

**Remark 14.** One interesting question is how Alg. 3 compares with FaSTrack [16] on a time-invariant system example. Note that the primary difference between Simulations 1 and 2 is that the former can “teleport”, which was pointed out in Remark 6 as the significant difference between Alg. 3 and FaSTrack (outside of the former supporting time-varying systems). Thus, we expect that implementing Alg. 3 similarly to Simulation 2 for a time-invariant system should produce faster goal-reaching compared to FaSTrack.

### E. Periodic Replanning and Tracking for the AUV

In this section, we demonstrate an implementation of Alg. 4 on a simulation of the same AUV subject to plane-progressive waves as in Sec. VII-C and VII-D. We aim to control the robot starting from  $s_0 = (-3.3, 2.7, 0.0, 0.0)$  so that it avoids all obstacles in the environment, which are *a priori* unknown, and reaches successive goal regions  $\mathcal{G}_1$ ,  $\mathcal{G}_2$ , and  $\mathcal{G}_3$ , which are illustrated in Fig. 7. In particular, as soon as the AUV hits  $\mathcal{G}_1$ , the active goal is changed to  $\mathcal{G}_2$ , and then when it hits  $\mathcal{G}_2$ , the active goal becomes  $\mathcal{G}_3$ . Unlike the previous example, we enlarge the environment to the region  $\{(x, z) \in \mathbb{R}^2 \mid x \in [-4, 4], z \in [2, 6]\}$ . To accommodate this large environment and varying goals, we allow  $T_{\text{run}} = 60$  seconds, set  $T_{\text{off}} = 14$  seconds, and  $T' = 4$  seconds. Notice that  $T_{\text{run}} \gg T_{\text{off}}$ . The sensor range of the AUV is 1.2 m, and again, the online, updated obstacles  $\mathcal{O}(t)$  represents known obstacle information at time  $t$ . The inputs for OfflineStage in line 2, as well as the implementation of PlanTrajectoryOverInterval in line 15, both correspond to Case 2, as described in VII-B.2 in VII-C.1.b respectively.

In addition to replanning when  $t - t_k \geq T'$ , we choose to replan whenever a new obstacle enters the sensor range of the AUV, as well as the moment it hits the current active goal region. We also fix  $c_k = 0.71$  and set  $p_k$  as the closest point in  $\mathcal{T}_p(s_k, t_k; c_k)$  to  $\mathcal{G}_p(t_k)$  which avoids  $\mathcal{O}_p(t_k)$ . Note that we relax the planning algorithm so that C4 is not enforced in Alg. 2 by removing the MPC goal constraint in (39) since the planning horizon is too small to plan a trajectory that reaches the planning system goal region.

Snapshots of the simulation at time instances where the active goal region has been reached are shown in Fig. 7. The AUV successfully avoids all obstacles and reaches all of the active goal regions in succession, despite the environment being *a priori* unknown. Additionally, the algorithm is successfully applied up until the final active goal is reached at  $t = 52.25$  seconds, which is significantly longer than  $T_{\text{off}} = 14$  seconds. This is achieved since Alg. 4 exploits the periodic properties of the system.

## VIII. CONCLUSION

In this paper, we proposed a method for safety-guaranteed trajectory replanning and tracking for generally time-varying systems while satisfying constraints updated online over a finite horizon. Our procedure is promising for real-time applications and allows planned trajectories to “teleport”, enabling flexible performance across diverse environments. This was modified to obtain an algorithm applicable over arbitrarily long time intervals for periodic systems. We analyze the constraint satisfaction and goal-reaching guarantees for these methods and demonstrate their utility on an example AUV subject to wave disturbances. We showed that modeling such systems as time-varying and using our replanning procedure allows the planning of less conservative safety-guaranteed trajectories for



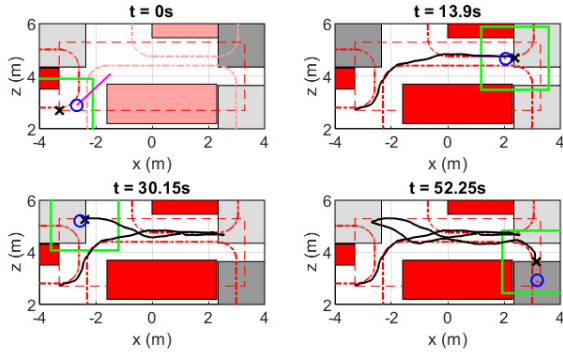


Fig. 7: Snapshots of the tracking and planning systems at time instances where the replanning occurs due to the AUV reaching the active goal region. The meaning of symbols and lines are consistent with Fig. 4, with the difference being that the dark grey box is the current active goal region, and the light grey boxes are the inactive goals.

faster goal-reaching. We demonstrated that in our replanning method, planning system teleportation can be exploited to trade off between faster goal-reaching versus feasible navigation through tight environments. Finally, we demonstrated that our replanning procedure can be used to safely control the AUV over arbitrarily long time intervals.

Although real-time performance is aided by the offline computation of the optimal tracking controller and the support for simple planning models, further work is required for concretely implementing online planning in Alg. 2 efficiently in specific applications, as it is the main computational bottleneck. Using the AUV case study as an example, it would be interesting to analyze the time complexity of the Case 1 obstacle computation procedure in Alg. 5 or determine a more efficient procedure. Moreover, it would be useful to improve the solver for the trajectory optimization problem. We leave these considerations to future work.

## REFERENCES

- [1] C. Badue, R. Guidolini, R. V. Carneiro, P. Azevedo, V. B. Cardoso, A. Forechi, L. Jesus, R. Berriel, T. M. Paixao, F. Mutz, et al., "Self-driving cars: A survey," *Expert Sys. Appl.*, vol. 165, p. 113816, 2021.
- [2] S. M. Adams and C. J. Friedland, "A survey of unmanned aerial vehicle (uav) usage for imagery collection in disaster research and management," in *9th Int. Workshop Remote Sens. Disaster Response*, vol. 8, pp. 1–8, 2011.
- [3] J. B. Paduan, D. W. Caress, D. A. Clague, C. K. Paull, H. Thomas, F. Chiocci, D. Ridente, D. Casalbone, and A. Bosman, "High-resolution mapping of mass wasting, tectonic, and volcanic hazards using the mbari mapping auv," in *Int. Conf. Seafloor Mapping Geohazard Assessment*, vol. 7, pp. 181–186, 2009.
- [4] A. Majumdar and R. Tedrake, "Funnel libraries for real-time robust feedback motion planning," *Int. J. Robot. R.*, vol. 36, no. 8, pp. 947–982, 2017.
- [5] S. W. Smith, H. Yin, and M. Arcak, "Continuous abstraction of nonlinear systems using sum-of-squares programming," in *Proc. IEEE Conf. Decis. Contr.*, pp. 8093–8098, IEEE, 2019.
- [6] S. Singh, M. Chen, S. L. Herbert, C. J. Tomlin, and M. Pavone, "Robust tracking with model mismatch for fast and safe planning: an sos optimization approach," in *Int. Workshop Algorithmic Found. Robot.*, pp. 545–564, Springer, 2018.
- [7] S. Kousik, S. Vaskov, F. Bu, M. Johnson-Roberson, and R. Vasudevan, "Bridging the gap between safety and real-time performance in receding-horizon trajectory design for mobile robots," *Int. J. Robot. R.*, vol. 39, no. 12, pp. 1419–1469, 2020.
- [8] H. Yin, M. Bujarbaruah, M. Arcak, and A. Packard, "Optimization based planner-tracker design for safety guarantees," in *Proc. Amer. Contr. Conf.*, pp. 5194–5200, IEEE, 2020.
- [9] K. S. Schweidel, H. Yin, S. W. Smith, and M. Arcak, "Safe-by-design planner-tracker synthesis with a hierarchy of system models," *Annu. Rev. Contr.*, 2022.

- [10] X. Xu, P. Tabuada, J. W. Grizzle, and A. D. Ames, "Robustness of Control Barrier Functions for Safety Critical Control," *IFAC-PapersOnLine*, vol. 48, pp. 54–61, 1 2015.
- [11] U. Rosolia and A. D. Ames, "Multi-rate control design leveraging control barrier functions and model predictive control policies," *IEEE Contr. Sys. Lett.*, vol. 5, no. 3, pp. 1007–1012, 2020.
- [12] S. Singh, H. Tsukamoto, B. T. Lopez, S.-J. Chung, and J.-J. Slotine, "Safe motion planning with tubes and contraction metrics," in *Proc. IEEE Conf. Decis. Contr.*, pp. 2943–2948, IEEE, 2021.
- [13] D. Q. Mayne, E. C. Kerrigan, E. J. Van Wyk, and P. Falugi, "Tube-based robust nonlinear model predictive control," *Int. J. Robust Nonlin. Contr.*, vol. 21, pp. 1341–1353, 7 2011.
- [14] A. Dhar, C. H. Ulfjoo, J. Lofberg, and D. Axehill, "Disturbance-parametrized robust lattice-based motion planning," *IEEE Trans. Intell. Vehicles*, 2023.
- [15] S. L. Herbert, M. Chen, S. Han, S. Bansal, J. F. Fisac, and C. J. Tomlin, "FaSTrack: A modular framework for fast and guaranteed safe motion planning," *Proc. IEEE Conf. Decis. Contr.*, vol. 2018-Janua, pp. 1517–1522, 2018.
- [16] M. Chen, S. L. Herbert, H. Hu, Y. Pu, J. F. Fisac, S. Bansal, S. Han, and C. J. Tomlin, "FaSTrack: a Modular Framework for Real-Time Motion Planning and Guaranteed Safe Tracking," *IEEE Trans. Autom. Contr.*, 2 2021.
- [17] D. Fridovich-Keil, S. L. Herbert, J. F. Fisac, S. Deglurkar, and C. J. Tomlin, "Planning, Fast and Slow: A Framework for Adaptive Real-Time Safe Trajectory Planning," *Proc. IEEE Int. Conf. Robot. Automat.*, pp. 387–394, 9 2018.
- [18] S. Siriya, M. Bui, A. Shriraman, M. Chen, and Y. Pu, "Safety-Guaranteed Real-Time Trajectory Planning for Underwater Vehicles in Plane-Progressive Waves," in *Proc. IEEE Conf. Decis. Contr.*, vol. 2020-December, pp. 5249–5254, Institute of Electrical and Electronics Engineers Inc., 12 2020.
- [19] E. A. Coddington and N. Levinson, *Theory of Ordinary Differential Equations*. McGraw-Hill, 1955.
- [20] I. M. Mitchell, A. M. Bayen, and C. J. Tomlin, "A time-dependent Hamilton-Jacobi formulation of reachable sets for continuous dynamic games," *IEEE Trans. Autom. Contr.*, vol. 50, pp. 947–957, 7 2005.
- [21] J. F. Fisac, M. Chen, C. J. Tomlin, and S. Shankar Sastry, "Reach-avoid problems with time-varying dynamics, targets and constraints," in *Proc. 18th Int. Conf. Hybrid Sys. Comput. Contr. HSCC 2015*, pp. 11–20, Association for Computing Machinery, Inc, 4 2015.
- [22] HJReachability, "Hjreachability/helperoc."
- [23] M. Bui, G. Giovanis, M. Chen, and A. Shriraman, "Optimizeddp: An efficient, user-friendly library for optimal control and dynamic programming," *arXiv preprint arXiv:2204.05520*, 2022.
- [24] A. Sahraekhanghah and M. Chen, "Pa-fastrack: Planner-aware real-time guaranteed safe planning," in *Proc. IEEE Conf. Decis. Contr.*, pp. 2129–2136, IEEE, 2021.
- [25] D. Bertsekas, *Dynamic programming and optimal control: Volume I*, vol. 1. Athena scientific, 2012.
- [26] M. Chen, S. L. Herbert, M. S. Vashishtha, S. Bansal, and C. J. Tomlin, "Decomposition of reachable sets and tubes for a class of nonlinear systems," *IEEE Trans. Autom. Contr.*, vol. 63, no. 11, pp. 3675–3688, 2018.
- [27] D. C. Fernandez and G. A. Hollinger, "Model Predictive Control for Underwater Robots in Ocean Waves," *IEEE Robot. Automat. Lett.*, vol. 2, no. 1, pp. 88–95, 2017.
- [28] T. Battista, C. Woolsey, L. McCue-Weil, E. Paterson, and F. Valentinis, "Underwater vehicle depth and attitude regulation in plane progressive waves," *Proc. IEEE Conf. Decis. Contr.*, vol. 54rd IEEE, no. Cdc, pp. 4400–4405, 2015.
- [29] X. Zhang, A. Liniger, and F. Borrelli, "Optimization-based collision avoidance," *IEEE Trans. Contr. Sys. Technology*, vol. 29, no. 3, pp. 972–983, 2020.
- [30] I. M. Mitchell and J. A. Templeton, "A toolbox of Hamilton-Jacobi solvers for analysis of nondeterministic continuous and hybrid systems," in *Lecture Notes Comput. Sci.*, vol. 3414, pp. 480–494, Springer, Berlin, Heidelberg, 2005.
- [31] A. A. Proctor, *Semi-Autonomous Guidance and Control of a Saab SeaEye Falcon ROV*. PhD thesis, University of Victoria, 2014.
- [32] A. Wächter and L. T. Biegler, "On the implementation of an interior-point. filter line-search algorithm for large-scale nonlinear programming," *Math. Program.*, vol. 106, pp. 25–57, 2006.

## APPENDIX I

### PROOF OF THEORETICAL RESULTS

*Proof of Prop. 1.* Suppose  $t' \in [t, T_{\text{off}}]$ . Then,  $V(r, t) = \max_{\tau \in [t, T_{\text{off}}]} l(\xi_g^*(\tau; r, t)) = \max \left\{ \max_{\tau \in [t, t']} l(\xi_g^*(\tau; r, t)), \max_{\tau \in [t', T_{\text{off}}]} l(\xi_g^*(\tau; \xi_g^*(t'; r, t), t')) \right\} \geq \max_{\tau \in [t', T_{\text{off}}]} l(\xi_g^*(\tau; \xi_g^*(t'; r, t), t')) = V(\xi_g^*(t'; r, t), t'). \quad \square$

*Proof of Lem. 2.* Suppose  $\xi_h^*(t'; p, t) \in \mathcal{F}_B(t'; c)$  holds. Since  $p \in \mathcal{T}_p(s, t_i; c)$ ,  $V(Ls - Mp, t) \leq c$  holds. Letting  $r = Ls - Mp$ , it

follows from Prop. 1 that for all  $t' \in [t, T_{\text{off}}]$ ,  $V(\xi_g^*(t'; r, t), t') \leq c$  holds. This can be equivalently expressed as

$$\begin{aligned} & V(\xi_g^*(t'; r, t), t') \leq c \\ \iff & V(L\xi_f^*(t'; s, t) - M\xi_h^*(t'; p, t), t') \leq c \\ \iff & \xi_f^*(t'; s, t) \in \{s' : V(Ls' - M\xi_h^*(t'; p, t), t') \leq c\} \\ \iff & \xi_f^*(t'; s, t) \in \mathcal{T}_s(t', \xi_h^*(t'; p, t); c), \end{aligned}$$

where  $\mathcal{T}_s$  is defined as

$$\mathcal{T}_s(p, t; c) := \{s : V(Ls - Mp, t) \leq c\} \subseteq \mathcal{S}$$

and is nonempty since  $c \geq \underline{V}(s, t)$ .

We now aim to prove  $\mathcal{T}_s(\xi_h^*(t'; s, t), t'; c) \subseteq B$ . Suppose  $s' \in \mathcal{T}_s(\xi_h^*(t'; s', t), t'; c)$ , such that  $V(Ls' - M\xi_h^*(t'; p, t), t') \leq c$ . We proceed by contradiction to prove that  $s' \in B$ . In particular, assume  $s' \in B^c$ . Since  $\xi_h^*(t'; p, t) \in \mathcal{F}_B(t'; c)$ , then  $V(Ls' - M\xi_h^*(t'; p, t)) > c$  follows, which contradicts  $V(Ls' - M\xi_h^*(t'; p, t), t') \leq c$ , and thus  $s' \in B$  holds. Therefore,  $\mathcal{T}_s(\xi_h^*(t'; p, t), t'; c) \subseteq B$  holds.

Since  $\xi_f^*(t'; s, t) \in \mathcal{T}_s(\xi_h^*(t'; p, t), t'; c)$ , and  $\mathcal{T}_s(\xi_h^*(t'; p, t), t'; c) \subseteq B$ , the conclusion follows.  $\square$

*Proof of Thm. 3.* Suppose  $t \in [t_i, t_f]$ , and  $\xi_h^*(t; p, t_i) \in \mathcal{F}_C(t; c)$ . Since (18) and (19) hold, applying Lem. 2, it follows that  $\xi_f^*(t; p, t_i) \in \mathcal{C}$ , establishing (19). Similarly,  $\xi_h^*(t; p, t_i) \in \mathcal{G}_p(t; c)$  implies  $\xi_f^*(t; s, t_i) \in \mathcal{G}$  via Lem. 2.  $\square$

*Proof of Cor. 4.* Suppose  $k \in \{0, \dots, K-1\}$ , and  $t \in [t_k, t_{k+1}]$ .

We first prove (26). Since  $\xi_h^R(t) \in \mathcal{F}_{C(t_k)}(t; c_k)$ , then by definition in (22) and the fact that  $s_k = \xi_f^R(t_k)$  (following induction), we know that  $\xi_h^*(t; p_k, \xi_f^R(t_k), t_k) = \xi_h^*(t; p_k, s_k, t_k) \in \mathcal{F}_{C(t_k)}(t; c_k)$ . Moreover, since  $c_k \geq \underline{V}(\xi_f^R(t_k), t_k)$  and  $p_k \in \mathcal{T}_p(\xi_f^R(t_k), t_k; c_k)$ , following (19) in Thm. 3,  $\xi_f^*(t; \xi_f^R(t_k), p_k, t_k) \in \mathcal{C}(t_k) = \mathcal{C}(t)$  holds for  $t \in [t_k, t_{k+1}]$ . Finally, since

$$\begin{aligned} \xi_f^R(t) &= \xi_f(t; s_0, 0, u_s^R(\cdot), d^R(\cdot)) \\ &= \xi_f(t; \xi_f^R(t_k), t_k, u_s^R(\cdot), d^R(\cdot)) \\ &= \xi_f(t; \xi_f^R(t_k), t_k, u_s^*(\cdot; L\xi_f^R(t_k) - Mp_k, t_k), \\ &\quad d^*(\cdot; L\xi_f^R(t_k) - Mp_k, t_k)) \\ &= \xi_f^*(t; \xi_f^R(t_k), p_k, t_k) \end{aligned}$$

it follows that  $\xi_f^R(t) \in \mathcal{C}(t)$ .

We now prove (27) in a similar manner. Since  $\xi_h^R(t) \in \mathcal{G}(t; c_k)$ , then by definition in (22) and  $s_k = \xi_f^R(t_k)$  we find  $\xi_h^*(t; p_k, \xi_f^R(t_k), t_k) = \xi_h^*(t; p_k, s_k, t_k) \in \mathcal{G}_p(t; c_k)$ . Moreover, since  $c_k \geq \underline{V}(\xi_f^R(t_k), t_k)$  and  $p_k \in \mathcal{T}_p(\xi_f^R(t_k), t_k; c_k)$ , following (20) in Thm. 3,  $\xi_f^*(t; \xi_f^R(t_k), p_k, t_k) \in \mathcal{G}$ .  $\square$

*Proof of Cor. 5.* Suppose  $k \in \{0, \dots, K-1\}$ , and  $t \in [t_k, t_{k+1}]$ .

We first prove (33). Since  $\xi_h^R(t) \in \mathcal{F}_{C(t_k)}(\mathcal{M}_\tau^f(t_k, t); c_k)$ , then by definition in (29) and the fact that  $s_k = \xi_f^R(t_k)$  (following induction) we know that  $\xi_h^*(\mathcal{M}_\tau^f(t_k, t); p_k, \xi_f^R(t_k), \mathcal{M}_\tau^i(t_k, t)) = \xi_h^*(\mathcal{M}_\tau^f(t_k, t); p_k, s_k, \mathcal{M}_\tau^i(t_k, t)) \in \mathcal{F}_{C(t_k)}(\mathcal{M}_\tau^f(t_k, t); c_k)$ . Moreover, since  $c_k \geq \underline{V}(\xi_f^R(t_k), \mathcal{M}_\tau^i(t_k, t_{k+1}))$  and  $p_k \in \mathcal{T}_p(\xi_f^R(t_k), \mathcal{M}_\tau^i(t_k, t_{k+1}); c_k)$ , following (19) in Thm. 3,  $\xi_f^*(\mathcal{M}_\tau^f(t_k, t); \xi_f^R(t_k), p_k, \mathcal{M}_\tau^i(t_k, t)) \in \mathcal{C}(t_k) = \mathcal{C}(t)$  holds for

$t \in [t_k, t_{k+1}]$ , where Thm. 3 can be applied since (30) implies  $\mathcal{M}_\tau([t_k, t_{k+1}]) \subseteq [0, T_{\text{off}}]$ . Finally, since

$$\begin{aligned} \xi_f^R(t) &= \xi_f(t; s_0, 0, u_s^R(\cdot), d^R(\cdot)) \\ &= \xi_f(t; \xi_f^R(t_k), t_k, u_s^R(\cdot), d^R(\cdot)) \\ &= \xi_f(t; \xi_f^R(t_k), t_k, \\ &\quad u_s^*(\mathcal{M}_\tau^f(t_k, \cdot); L\xi_f^R(t_k) - Mp_k, \mathcal{M}_\tau^i(t_k, \cdot)), \\ &\quad d^*(\mathcal{M}_\tau^f(t_k, \cdot); L\xi_f^R(t_k) - Mp_k, \mathcal{M}_\tau^i(t_k, \cdot))) \\ &= \xi_f^*(\mathcal{M}_\tau^f(t_k, t); \xi_f^R(t_k), p_k, \mathcal{M}_\tau^i(t_k, t)), \end{aligned}$$

it follows that  $\xi_f^R(t) \in \mathcal{C}(t)$ .

We now prove (34) in a similar manner. Since  $\xi_h^R(\mathcal{M}_\tau^f(t_k, t)) \in \mathcal{G}(\mathcal{M}_\tau^f(t_k, t); c_k)$ , then by definition in (29) and  $s_k = \xi_f^R(t_k)$  we know that  $\xi_h^*(\mathcal{M}_\tau^f(t_k, t); p_k, \xi_f^R(t_k), \mathcal{M}_\tau^i(t_k, t)) = \xi_h^*(\mathcal{M}_\tau^f(t_k, t); p_k, s_k, \mathcal{M}_\tau^i(t_k, t)) \in \mathcal{G}_p(\mathcal{M}_\tau^f(t_k, t); c_k)$ . Moreover, since  $c_k \geq \underline{V}(\xi_f^R(t_k), \mathcal{M}_\tau^i(t_k, t))$  and  $p_k \in \mathcal{T}_p(\xi_f^R(t_k), \mathcal{M}_\tau^i(t_k, t); c_k)$ , following (20) in Thm. 3,  $\xi_f^R(t) = \xi_f^*(\mathcal{M}_\tau^f(t_k, t); \xi_f^R(t_k), p_k, \mathcal{M}_\tau^i(t_k, t)) \in \mathcal{G}$  holds.  $\square$



**Seth Siriya** received the B.Sc. degree in electrical systems, in 2016, and the M.Eng. degree in electrical engineering, in 2019, from the University of Melbourne, Parkville, Australia, where he is currently working toward the Ph.D. degree in electrical engineering.

His research interests include safe control for robotics, and learning-based control.



**Mo Chen** is an Assistant Professor in the School of Computing Science at Simon Fraser University, Burnaby, BC, Canada, where he directs the Multi-Agent Robotic Systems Lab. He holds a Canada CIFAR AI Chair position and is an Amii Fellow. Dr. Chen completed his PhD in the Electrical Engineering and Computer Sciences Department at the University of California, Berkeley in 2017, and received his B.A.Sc. in Engineering Physics from the University of British Columbia in 2011. From 2017 to 2018, He was a postdoctoral researcher in the Aeronautics and Astronautics Department in Stanford University. Dr. Chen's research interests include multi-agent systems, safety-critical systems, human-robot interactions, control theory, reinforcement learning, and their intersections.



**Ye Pu** Ye Pu earned a B.S. degree from Shanghai Jiao Tong University, China, in 2008, an M.S. degree from the Technical University Berlin, Germany, in 2011, and a Ph.D. degree in Electrical Engineering from the Swiss Federal Institute of Technology (EPFL), Lausanne, Switzerland, in 2016. Currently, she holds the position of a Senior Lecturer and an ARC DECRA fellow in the Department of Electrical and Electronic Engineering at the University of Melbourne, Australia. Prior to that, she worked as a Postdoctoral Scholar in the Hybrid Systems Lab, EECS, at the University of California at Berkeley, USA from 2016-2018.